



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A szoftveralkalmazás dokumentációja

Szakember Kereső

Készítették:

CSÁSZÁR BÁLINT

VERHÁS GERGŐ

VARGA DÁVID TIBOR

SZOMBATHELY

2026

Tartalom

1	Bevezetés	5
1.1	Projektünk alapötlete	5
1.2	Az elkészült szoftver rövid ismertetése	6
2	A szoftver célja	7
2.1	Felhasználói szerepkörök	7
2.1.1	Általános felhasználó	7
2.1.2	Szakember	7
2.2	Az alkalmazás előnyei	7
3	Megvalósítás	8
3.1	Tervezés	8
3.2	Adatbázis	8
3.3	Frontend rövid áttekintése	9
3.4	Backend rövid áttekintése	9
3.5	A fejlesztéshez használt eszközök	10
4	Adatbázis-terv	10
4.1	Rövid összefoglaló	10
4.2	ER-diagram	10
4.3	Entitások részletes leírása	10
4.3.1	users	10
4.3.2	experts	11
4.3.3	occupations	11
4.3.4	expert_specialties	12
4.3.5	pricing	12
4.3.6	services	12
4.3.7	jobs	13
4.3.8	time_intervals	14
4.3.9	settlements	14
4.3.10	residential_addresses	14
4.3.11	ASP.NET Identity segédtablák	15
4.4	Kapcsolatok levezetése	15
4.4.1	users – experts (1:1)	15
4.4.2	experts - occupations (N:M)	15
4.4.3	expert_specialties – services (1:N)	15
4.4.4	services – jobs (1:N)	16
4.4.5	users – jobs (1:N)	16

4.4.6	jobs - time_intervals (1:N).....	16
4.4.7	services – pricing (1:1)	16
4.4.8	jobs – pricing (1:1).....	16
4.4.9	residential_addresses – jobs (1:N)	16
4.4.10	settlements - residential_addresses (1:N)	17
4.4.11	settlements – experts (1:N)	17
5	A komponensek technikai leírása	17
5.1	Backend - ASP.NET Core Web API	17
5.1.1	Az API felépítése	17
5.1.2	Autentikáció és autorizáció.....	18
5.1.3	Végpontok ismertetése.....	18
5.1.4	DTO-k, Mapster és FluentValidation.....	22
5.2	Frontend - Next.js alkalmazás	23
5.2.1	Next.js használata	23
5.2.2	Server Side Rendering	24
5.2.3	Material UI.....	25
5.2.4	Egyedi React hook-ok és SWR.....	25
6	Működés műszaki feltételei	26
6.1	Szükséges szoftverek	26
6.2	A projekt mappaszerkezete.....	26
6.3	Adatbázis előkészítése	26
6.3.1	Adatbázis dump fájlok	26
6.3.2	Futtatás – Dockert használatával	27
6.3.3	Futtatás – Natív MySQL telepítés.....	27
6.3.4	Elérhető települések (fő dump).....	28
6.3.5	Előre meghatározott felhasználói fiókok	28
6.4	Futtatás – Production környezet	28
6.4.1	Backend indítása	28
6.4.2	Frontend indítása.....	28
6.5	Futtatás – Fejlesztői környezet	29
6.5.1	Adatbázis indítása	29
6.5.2	Backend indítása	29
6.5.3	Frontend indítása.....	29
7	Az alkalmazás működése.....	30
7.1	Kezdő oldal.....	30
7.1.1	Az oldal felépítése	30
7.1.2	Keresés indítása	30

7.2	Bejelentkezés, regisztráció	31
7.2.1	Bejelentkezés	31
7.2.2	Regisztráció	31
7.2.3	Regisztráció szakemberként	32
7.2.4	Regisztráció általános felhasználóként	32
7.3	Kereső oldal	33
7.3.1	Az oldal felépítése	33
7.3.2	Szűrő panel	33
7.3.3	Találatok panel	34
7.3.4	URL	36
7.4	Navigációs menü	36
7.5	Szakember publikus profilja	37
7.5.1	Az oldal felépítése	37
7.6	Dashboard	38
7.6.1	Az oldal felépítése	38
7.6.2	Profil beállítások oldal	38
7.6.3	Foglalkozások oldal	39
7.6.4	Szolgáltatások oldal	39
7.6.5	Munkák oldal	40
8	Teszt dokumentáció	43
8.1	Rendszertesztek	43
8.2	Egység tesztek	43
8.2.1	AuthController	43
8.2.2	ExpertsController	43
8.2.3	JobsController	43
8.2.4	OccupationsController	44
8.2.5	ServicesController	44
8.2.6	SettlementsController	44
8.2.7	UsersController	44
9	Mellékletek	45
9.1	7-5. sz. táblázat	45
9.2	Er-diagram	46
9.3	Fő dump települései	46
9.4	Felhasználók listája	46
9.5	Egységtesztek – AuthController	47
9.6	Egységtesztek – ExpertsController	48
9.7	Egységtesztek – JobsController	48

9.8	Egységtesztek – OccupationsController.....	51
9.9	Egységtesztek – ServicesController	51
9.10	Egységtesztek – SettlementsController	51
9.11	Egységtesztek – UsersController.....	52

1 Bevezetés

1.1 Projektünk alapötlete

A projekt ötlete abból a problémából született, hogy napjainkban egyre nagyobb kihívást jelent megbízható és hozzáértő szakembereket találni otthoni javítási, felújítási vagy egyéb szakképesítést igénylő munkákhoz. Az emberek jellemzően ismerőseik ajánlásaira támaszkodnak, vagy böngésznek az interneten. Mindkét módszer időigényes és bizonytalan kimenetelű. Ugyanakkor a szakemberek oldaláról nézve szintén problémát jelent az ügyfelekkel való kapcsolatteremtés. A hagyományos hirdetési csatornákon keresztül nehéz célzottan elérni az érdeklődő megbízókat.

A probléma szemléltetésére vegyünk egy hétköznapi példát. Tegyük fel, hogy mi, mint továbbtanulás előtt álló fiatalok, Győrbe egy albérletbe költözünk. Egy hét elteltével arra leszünk figyelmesek, hogy a tusolóból nem folyik a melegvíz. Feltételezzük, hogy a bojler hibásodott meg. Szükségünk lenne egy szakemberre, aki a felmerült problémánkra megoldást nyújtana. Mivel újak vagyunk a városban, nem tudjuk, hogy hol és milyen eszközök segítségével keressük meg a megfelelő vállalkozót. Eddig rokonokon, barátokon keresztül, volt egy „már bevált”, „körzeti” szakember, akit fel lehetett keresni és meg lehetett bízni a munkával. Ezúttal, ez nem opció, így más módszerekhez kell nyúlnunk. Például Facebookon böngészünk, hirdetéseket keresünk, helyi kisipari üzletekben érdeklődünk. Tegyük fel, hogy hosszas keresés után, kaptunk egy telefonszámot. Felhívtuk a szakembert, elmondtuk a problémánkat és időpontot egyeztettünk. Egy hét múlva elérkezett a megbeszéltek nap, de a mesterember nem jelent meg. Újból felhívtuk, közölte velünk, hogy sajnálatosan megfeledkezett a megbeszéltekről. Ismét időpontot egyeztettünk, és ezúttal meg is jelent. Megvizsgálta az elromlott bojler, és megállapította, hogy a probléma megjavításához, cserealkatrészre lesz szükség, amit meg kell rendelnie. Újabb időpont egyeztetés, újabb várakozás, és csak hosszabb idő elteltével lesz újra meleg víz.

A probléma megközelíthető a másik oldalról is, a szakember oldaláról. A megbízások különféle csatornákon keresztül érkehetnek, például telefonos egyeztetés, szóbeli megállapodás, emailben vagy online felületen történő megkeresés (pl. Messenger). Mindezt nyomon követni és rendszerezni nem egyszerű feladat.

A Szakember Kereső ezen kihívásokra kínál megoldást azzal, hogy egy platformon hozza össze a munkát kereső szakembereket és az ő szolgáltatásaikat igénybe vevő megbízókat.

1.2 Az elkészült szoftver rövid ismertetése

A Szakember Kereső egy webes alkalmazás, amelynek segítségével felhasználók megtalálhatják a számukra megfelelő szakembert. Áttekinthetik az általuk nyújtott szolgáltatásokat, és azok részleteit. Gyors és érintésmentes megbízásokat kezdeményezhetnek. A mesteremberek saját profilt kezelhetnek, mely lehetőséget ad foglalkozások hozzáadására, valamint azokhoz kapcsolódó különféle szolgáltatások kínálására. Ezen keresztül láthatják a beérkező megkereséseket, melyekre ajánlatot adhatnak. Az elfogadott munkákat időpontjait és folyamatát egyszerűen nyomon tudják követni.

2 A szoftver célja

2.1 Felhasználói szerepkörök

A bevezetésben felvázoltakból látható, hogy alkalmazásunk két külön felhasználói csoport problémáinak megoldására irányul: az *általános felhasználók* és a *szakemberek*.

2.1.1 Általános felhasználó

Az általános/közönséges felhasználó kifejezés alatt munkát, illetve szolgáltatást kereső magánszemélyt értjük. Egy ilyen kliens az alkalmazásunkon keresztül:

- böngészheti a regisztrált szakemberek profilját, szakterületeit és díjszabását,
- munkamegbízást indíthat egy kiválasztott szakember adott szolgáltatására,
- megtekintheti és elfogadhatja a szakember által küldött árajánlatot és időpont-javaslatokat,
- az elvégzett munka után értékelést adhat.

2.1.2 Szakember

A szakember egy olyan regisztrált felhasználó, aki valamilyen szaktudással, szakképesítéssel rendelkezik. Alkalmazásunkon keresztül:

- részletes szakmai profilt hozhat létre, megadhatja szakterületeit és azokhoz szolgáltatásokat rendelhet,
- meghatározhatja az általa nyújtott szolgáltatásokat és azok díjszabását (*fix díj*, illetve *fix alap + egységár* ár modellek)
- saját irányítópultján kezeli a beérkező munkamegbízásokat,
- árajánlatot adhat, javasolt időpontokat jelölhet meg,
- az elvállalt munkákat elvégzettnek jelölheti.

2.2 Az alkalmazás előnyei

Az általunk fejlesztett alkalmazás számos előnyt kínál mind a felhasználók, mind a szakemberek számára. A rendszer egyik legfontosabb célja, hogy egyszerűbbé, gyorsabbá és átláthatóbbá tegye a szakemberkeresés és a szolgáltatás igénylésének folyamatát.

Az alkalmazás egyik kiemelkedő előnye, hogy lehetővé teszi az érintésmentes ügyintézés. A felhasználók online kereshetnek szakembereket, böngészhetnek a lehetőségek

között, és kiválaszthatják a számukra legmegfelelőbb szolgáltatót anélkül, hogy személyes jelenlétre lenne szükség. Ez különösen előnyös gyors ügyintézés és kényelmes használat szempontjából.

További előny, hogy az alkalmazás ingyenes reklámfelületet biztosít a szakemberek számára. A rendszer lehetőséget ad arra, hogy a szakemberek bemutatkozzanak, megjelenítsék szolgáltatásaikat, és szélesebb körben elérhetővé váljanak a potenciális ügyfelek számára. Ez különösen hasznos lehet kisebb vállalkozások és egyéni szakemberek számára.

Az alkalmazás emellett gyors és hatékony keresési lehetőségeket biztosít, amelyek segítségével a felhasználók könnyen megtalálhatják a számukra megfelelő szakembereket. A szűrési és keresési funkciók révén időt takaríthatnak meg, és egyszerűbben hozhatnak döntést.

Összességében az alkalmazás célja, hogy modern, felhasználóbarát és hatékony megoldást nyújtson a szakemberkeresés területén, miközben mind a felhasználók, mind a szolgáltatók számára előnyös lehetőségeket biztosít.

3 Megvalósítás

3.1 Tervezés

A tervezési fázis a vizsgaremek specifikációs dokumentumának elolvasásával kezdődött. Tudtuk, hogy az elkészült munkának arra kell törekednie, hogy egy létező és jelenleg megoldatlan problémára adjon megoldást. Ennek tudatában, az alapötlet 2025. év tele és tavasza között alakult ki. A kezdetleges ötlet: egy webes felület készítése, ahova szakemberek regisztrálhatnak, különféle referenciákkal feltölthetik a publikus profil oldalukat, egyéb látogatók pedig azokat megtekinthetik. Az alkalmazás másik funkciója, az egyes munkák igénylése és azoknak elfogadása, digitális alapon történő megvalósításának ötlete csak később született.

3.2 Adatbázis

A projektmunka elkészítésének második fő része az adatbázis megtervezése és elkészítése volt. Az adatok tárolására MySQL 8-as adatbázist hoztunk létre. A relációs modell tökéletesen megfelelt a tervezett alkalmazás igényeinek kiszolgálására. A MySQL-t

azért választottuk, mivel az iskolai fejlesztői környezetben is leginkább ezt használtuk. Így magabiztosan tudtunk vele dolgozni. Az adatbázis futtatására Dockert használtunk. Az egyed típusok bemutatására egy későbbi fejezetben (4 Adatbázis-terv) kerül sor.

3.3 Frontend rövid áttekintése

Frontend: Next.js 16 keretrendszerrel, Material UI komponenskönyvtárral és SWR adatlekérési könyvtárral megvalósított, és szerver oldali renderelést is támogató webes felület.

Használt technológiák és fontosabb csomagok:

Csomag / technológia	Verzió	Szerepe
Next.js	16.1.1	React alapú keretrendszer, spéci, hasznos funkciókkal.
Material UI	7.3.7	Webes felülethez használt komponenskönyvtár.
MUI X Date Pickers	8.27.2	Dátum- és időpont-választó komponensek.
SWR	2.4.1	Szerver-állapot kezelése, adatlekérés és cache.
Day.js	1.11.19	Dátumok kezelése és formázása pár komponens estében.

3-1. táblázat

3.4 Backend rövid áttekintése

Backend: C#, ASP.NET Core (.NET 8) keretrendszer segítségével készült RESTful API. Az adatok tárolásához MySQL adatbázist, az objektum-relációs leképezéshez Entity Framework Core-t használtunk. ASP.NET Identity-t használtunk a felhasználók kezelésére.

Használt technológiák és fontosabb csomagok:

Csomag / technológia	Verzió	Szerepe
.NET	8.0	Futtatókörnyezet
ASP.NET Core	8.0	Webes alkalmazás kiszolgálására készült API keretrendszere
Entity Framework Core	8.0	ORM + migrációk
Pomelo.EntityFrameworkCore.MySql	9.0	EF Core MySQL-provider
ASP.NET Identity	8.0	Felhasználók regisztrálása, kezelése
Mapster	7.4	Adatátviteli objektumokkal való dolgozás megkönnyítése
FluentValidation	12.1	Bemeneti adatok validációja
Swashbuckle (Swagger)	6.6	Gyors és egyszerű API tesztelő UI

3-2. táblázat

3.5 A fejlesztéshez használt eszközök

Terület	Eszközök
Backend	.NET 8 SDK, Visual Studio 2022 Community Edition
Frontend	Node.js 22, npm, Visual Studio Code
Adatbázis	Docker, Docker Desktop, phpMyAdmin
Csapatmunka	Git, GitHub

4 Adatbázis-terv

4.1 Rövid összefoglaló

A rendszer adatait egy MySQL 8 relációs adatbázis tárolja. A séma Entity Framework Core Code-First megközelítéssel készült. Ez azt jelenti, hogy az adatbázis-táblákat a C# entitáosztályok és a *Context.cs*-ben definiált EF Core konfigurációk alapján a *dotnet-ef tool* migrációs rendszere hozta létre.

A rendszer összesen 10 alkalmazásspecifikus táblából áll, amelyek mellé az ASP.NET Identity keretrendszer automatikusan létrehozza a felhasználókezeléshez szükséges segéd táblákat. Ezeket a táblanevek előtti '_' karakter jelöli.

4.2 ER-diagram

Fontos megemlíteni, hogy az átláthatóság érdekében az elkészült ER-diagrammból szándékosan kihagytuk az ASP.NET Identity által létrehozott egyed típusokat. (lásd: 9.2 *Er-diagram*)

4.3 Entitások részletes leírása

4.3.1 users

Az alkalmazás minden regisztrált felhasználójának adatait tároló egyed típus. Az ASP.NET Identity csomag *IdentityUser<int>* osztályát terjeszti ki két saját mezővel.

Attribútumai:

- id (int, PK): Automatikusan generált azonosító.
- FirstName (varchar(32)): A felhasználó keresztnéve.
- LastName (varchar(32)): A felhasználó vezetéknéve.
- Email (varchar(256)): A bejelentkezéskor megadott email cím.
- PhoneNumber (longtext): Opcionálisan megadható telefonszám.

- **UserName (varchar(256)):** Az ASP.NET Identity számára szükséges, az egyedtípusokat egyértelműen azonosító, egyedi mező. Alkalmazásunkban megegyezik az email attribútummal. Ez a mező felel azért, hogy minden email címhez kizárólag egy felhasználói profil tartozzon.
- **+ Identity mezők:** Az ASP.NET által létrehozott és annak működéséhez szükséges attribútumok (pl.: Biztonsági bélyeg, normalizált e-mail cím és username).

4.3.2 experts

Gyenge egyedtípus, önmagában nem létezhet, nincs értelme. Ez tárolja a szakember profilhoz szükséges „extra” attribútumokat.

Attribútumai:

- **user_id (int, PK, FK):** Hivatkozik egy *users* egyedre.
- **biography (varchar(512)):** A szakember által meghatározható, rövid bemutatkozó szöveg.
- **company_email (varchar(256)):** A vállalkozás email címe.
- **company_phone_number (varchar(32)):** A vállalkozás telefonszáma.
- **work_location_id (int (FK)):** A szakember elhelyezkedésének, munkavégzésének települése.

4.3.3 occupations

A FEOR (Foglalkozások Egységes Osztályozási Rendszere) szerinti foglalkozási kategóriák katalógusa. Az elsődleges kulcs maga a FEOR-kód. Fontos megjegyezni, hogy a fő dump file (*/database/vrdb_partial_dump.sql*) az összes lehetséges foglalkozás csak egy részét tartalmazza. Erről bővebben egy későbbi szekcióban lesz szó (*6.3.1 Adatbázis dump fájlok*).

Attribútumai:

- **feor_id (int, PK):** Az adott foglalkozást egyértelműen azonosító FEOR-kód (pl.: 7524 - Épületvillamossági szerelő, villanyszerelő).
- **name (varchar(256)):** A foglalkozás neve.
- **description (varchar(256)):** Az adott foglalkozás rövid leírása.

4.3.4 expert_specialties

Az *experts* és az *occupations* tábla közötti N:M kapcsolat megvalósítására szolgáló kapcsolótábla.

Attribútumai:

- *id* (int, PK): Egyedi azonosító.
- *expert_id* (int, FK): Hivatkozás az *experts* táblára.
- *occupation_id* (int, FK): Hivatkozás az *occupations* táblára.

4.3.5 pricing

Ez az egyedtípus tárolja az árazási beállításokat a *jobs* és a *services* entitásoknak. Látható, hogy van egy *pricing_type* attribútuma, ami a következő árazási modelleket támogatja: *fix díj* vagy *fix díj és egységár*. Ez a valóság leképezése szempontjából egy fontos döntés volt, hiszen vannak munkák, illetve szolgáltatások, amiket nem lehet egyszerűen egy előre meghatározott árral megjelölni. Vegyünk példaként egy festőt, aki valószínűleg négyzetméterben, vagy a teljesített munkaórák számában kívánja díjazni az elvégzett szolgáltatást. Ezt a logikát több helyen is alkalmaztuk, ezért egy külön egyedtípus formájában került megvalósításra.

Attribútumai:

- *pricing_id* (int, PK): Egyedi azonosító.
- *pricing_type* (tinyint): Tárolja az árazás típusát. 0 = Fix díj, 1 = Fix + egységár. Ennek alapját a projekt backend részén belül egy PricingType enum biztosítja.
- *fixed_price* (decimal): Az alapidjat reprezentálja, melynek meghatározása mindkét típusnál kötelező.
- *unit_price* (decimal): Az egységárat reprezentálja. Kötelező meghatározni abban az esetben, amikor a *pricing_type* 1-es értéket vesz fel.
- *unit_name* (varchar(64)): Az egység neve. Kötelező meghatározni abban az esetben, amikor a *pricing_type* 1-es értéket vesz fel.

4.3.6 services

Lehetőséget ad, egy a szakember valamely felvett szakmájához tartozó szolgáltatás meghatározására.

Attribútumai:

- `service_id` (int, PK): Egyedi azonosító.
- `name` (varchar(100)): A szolgáltatás megnevezése.
- `description` (varchar(512)): Opcionális leírás.
- `expert_specialties_id` (int, FK): Hivatkozás az *expert_specialties* táblára.
- `pricing_id` (int, FK): Hivatkozás a *pricing* táblára.

4.3.7 jobs

A munkafolyamat központi táblája. Egy munkamegbízás, a teljes életciklusa alatt, ebben a táblában kerül nyilvántartásra, a status oszlop változásán keresztül.

Egy munka életciklusának rövid bemutatása:

- 0 = Pending: Megbízás elküldve, szakember még nem válaszolt.
- 1 = Offered: Szakember árajánlatot adott.
- 2 = Accepted: A megbízó elfogadta az ajánlatot.
- 3 = Completed: Munka elvégezve.
- 4 = Cancelled: A megbízás lemondva.

A munka állapotát a backend kódban a *JobStatus* enum reprezentálja, melynek definíciója megtalálható a backend projekt *SzakemberKereso/Models/Job.cs* állományában.

Attribútumai:

- `job_id` (int, PK): Egyedi azonosító.
- `status` (tinyint): A munka állapotának meghatározása (0-4).
- `initiating_user_id` (int, FK): A megbízó felhasználó.
- `service_id` (int, FK): A megrendelt szolgáltatás.
- `location_id` (int, FK): Munkavégzés helyszíne. Hivatkozás a *residential_addresses* táblára.
- `title` (varchar(100)): Megbízást röviden összefoglaló attribútum, hasonlít egy email tárgy mezőjére. Azért van rá szükség, hogy a webes felületen egyszerűen be lehessen azonosítani egy adott munkákat.
- `description` (longtext): A megbízás részletes leírása. Opcionálisan elhagyható.
- `pricing_id` (int, FK): Az ajánlatban szereplő díjazás. Kezdetben null értéket vesz fel. Az ajánlat meghatározásakor kap értéket. Hivatkozás a *pricing* táblára.
- `cancelled_from_status` (tinyint): Segéd attribútum. A munka státuszának *Cancelled* állapotra állításakor van rá szükség, hogy később tudjuk melyik state lett lemondva.

- rating (float): A megbízó értékelése. A munka lezárása után kerülhet meghatározásra.

4.3.8 time_intervals

Egy munkához tartozó időpont-bejegyzések. Két típus létezik: a megbízó által megadott elérhetőség (*Available*) és a szakember által javasolt konkrét időpont (*Offered*).

Attribútumai:

- time_interval_id (int, PK): Egyedi azonosító.
- job_id (int, FK): Hivatkozás a jobs táblára.
- type (tinyint): 0 = Available, 1 = Offered.
- date (date): A meghatározott dátum.
- start_time (time): Kezdési időpont.
- end_time (time): Befejezési időpont.

4.3.9 settlements

Magyarország településeit tárolja. Referenciatáblaként működik, hasonlóan, mint az *occupations*. A backend nem módosítja az egyes rekordokat, nem ad hozzá újat, illetve nem törli azokat. Fontos megjegyezni, hogy a fő dump file (*/database/vrdb_partial_dump.sql*) az összes létező település csak egy részét tartalmazza. Erről bővebben egy későbbi szekcióban lesz szó (*6.3.1 Adatbázis dump fájlok*).

Attribútumai:

- settlement_id (int, PK): Egyedi azonosító.
- postal_code (int): Irányítószám.
- name (varchar(100)): Település neve.
- county_name (varchar(100)) Vármegye neve.

4.3.10 residential_addresses

Konkrét, utca szintű lakcímeket tároló tábla. Egy cím egy *settlement*hez tartozik.

Attribútumai:

- residential_address_id (int, PK): Egyedi azonosító.
- settlement_id (int, FK): Hivatkozás a settlements táblára.
- street_name (varchar(150)): Utca neve.
- public_area_type (varchar(16)): Közterület típusa (utca, tér, út, körút...).

- `building_number` (int): Házszám.

4.3.11 ASP.NET Identity segédtablák

Az ASP.NET Identity automatikusan létrehozza az alábbi segédtablákat. Ezek tartalma az alkalmazás által közvetlenül nem módosítható, az Identity keretrendszer kezeli azokat.

Röviden a táblákról:

- `roles`: Szerepkörök (Expert, Generic).
- `_user_roles`: Felhasználó–szerepkör hozzárendelések.
- `_user_claims`: Felhasználói claim-ek.
- `_user_logins`: Külső bejelentkezési szolgáltatók (jelenleg nem használt).
- `_user_tokens`: Biztonsági tokenek.
- `_role_claims`: Szerepköri claim-ek.

4.4 Kapcsolatok levezetése

4.4.1 `users` – `experts` (1:1)

A `users` táblában vannak jelen azok a mezők, melyek függetlenül attól, hogy általános felhasználói vagy szakember profilról beszélünk, mindkét esetben szükségesek. Így felesleges lett volna egy külön táblát létrehozni a vállalkozói profilok tárolására, amely nagyrészt megegyezett volna a másik táblával. Az *experts* egyed típus ezért csak a kiegészítő attribútumokat tárolja, amelyek egy szakember felhasználói fiókjához szükségesek (pl.: *work_location_id*). Az így létrejött gyenge egyed típus 1:1 kapcsolatban áll a *users* táblával.

4.4.2 `experts` - `occupations` (N:M)

A kapcsolat a valóságnak megfelel: egy szakember több szakterületen belül is tevékenykedhet, valamint egy foglalkozáson belül több vállalkozó is munkálkodhat. Ez a reláció az *expert_specialties* kapcsolótáblával lett megvalósítva.

4.4.3 `expert_specialties` – `services` (1:N)

A szakember szolgáltatásai, az ő általa megadott szakterületek alatt csoportosulnak. Minden ilyen kizárólag egy foglalkozás alá tartozhat. Ezért az 'N' oldalon a szolgáltatások, míg az '1' oldalon a foglalkozások állnak.

4.4.4 services – jobs (1:N)

A *jobs* egyed típus határozza meg a *service_id* attribútumot, így ez alkotja a kapcsolat 'N' oldalát. Jól látható, hogy a megrendelő, a munka létrehozásakor nem magát a szakembert kéri fel, hanem a szakember valamely szolgáltatására nyújt be kérelmet.

4.4.5 users – jobs (1:N)

A *jobs* egyed típus, meghatároz egy *initiating_user_id* mezőt, így minden munkának csak egy megrendelője lehet. Egy felhasználóhoz több munka is tartozhat.

4.4.6 jobs - time_intervals (1:N)

Egy munkához több idő intervallum is tartozhat. Először is a kezdeményezőnek meg kell határoznia legalább egy elérhetőségi időpontot, amikor számára alkalmas lenne a munka kivitelezése. A szakember pedig ajánlattételkor meghatározza, hogy melyek lennének a megfelelő időpontok.

4.4.7 services – pricing (1:1)

Az alkalmazásunkat használó szakember minden szolgáltatásához hozzá tud rendelni egy általa meghatározott árazást.

4.4.8 jobs – pricing (1:1)

A *jobs* egyed típus ugyan már kapcsolatban áll a *services* entitással, ami minden esetben meghatároz egy árazást, azonban a valóságból kiindulva, ezt nem vehetjük a munka tényleges árának. Előfordulhat, hogy a megrendelő drága festékekkel szeretné nappaliját kifestetni. A festőnek ezért lehetőséget kell adni, hogy módosítson a szolgáltatásban meghatározott árazáson. Az előző példát folytatva, ha 500Ft/m² helyett, a drágább anyagköltség miatt 1000Ft/m²-t szeretne megadni a szakember, a jelenlegi struktúra erre lehetőséget ad.

4.4.9 residential_addresses – jobs (1:N)

Az elkészült szoftver egy munkához egy kivitelezési helyszín hozzárendelését engedi meg. A valóságot nézve ez nem probléma, hiszen a kisebb javítási munkákra általában egy helyen, a megrendelő lakhelyén adódik igény. Egy lakcímhez több megrendelés is tartozhat, mivel az alkalmazás, a kérvényezési fázisban megnézi, hogy a megadott cím szerepel e már az adatbázisban. Abban az esetben, ha ez így van nem jön létre új rekord a *residential_addresses* táblában, a meglévő adatra fog irányulni a hivatkozás.

4.4.10 settlements - residential_addresses (1:N)

Az alkalmazás tervezési fázisában ez a két egyedtípus nem volt elkülönítve egymástól. Később azonban rájöttünk, hogy a felhasználó által megadott irányítószám és városnév validálására, illetve a kereső oldalon a megyékre történő szűrés érdekében, elengedhetetlen lesz egy külön adatbázis táblába feltölteni az összes lehetséges Magyarországi helységet. Az elkészült szoftverben így lett megvalósítva a címek kezelése: mindegyikhez szükséges hozzárendelni egy települést. Ez az adatok redundanciájának szempontjából is pozitív strukturális döntés volt, hiszen, ha több cím, egyazon településhez tartozik, akkor a helység adatai nem lesznek többszörösen tárolva.

4.4.11 settlements – experts (1:N)

Minden szakember esetében meg lehet határozni egy települést, ahol vállalkozása működik, szolgáltatásait ajánlja. Ennek segítségével tudjuk az alkalmazás kereső oldalán, adott helységre szűkíteni a keresés eredményét.

5 A komponensek technikai leírása

5.1 Backend - ASP.NET Core Web API

Csapatunk az adatbázis kezelésére, a szerveroldali logika megvalósítására és a frontendhez szükséges API végpontok kialakítására egy ASP.NET Core 8 alapú RESTful API-t készített.

5.1.1 Az API felépítése

REST architektúrára esett a választás, mivel a tervezés során felmértük, hogy az elkészítendő alkalmazás minden lehetséges igényét kielégítené. Döntő szerepet játszott az is, hogy a középiskolás tanulmányaink során hasonló felépítésű projekteken dolgoztunk, így minden csapattag tisztában volt a lehetőségekkel és korlátokkal, amiket ez a backend megközelítés kínált.

A .NET keretrendszer kiválasztása is egészen egyértelmű volt számunkra. Gondolkoztunk JavaScript alapú megvalósításon is, de végül egyetértettünk, hogy az általunk már jól ismert C# alapú framework, a nyelv erősen típusossága miatt, ideálisabb választás az alkalmazás üzleti logikájának megvalósítására.

A Microsoft által fejlesztett nyílt forráskódú *Entity Framework Core-t* használtuk a projekt ORM (*Object-Relational Mapping*) keretrendszereként. Ennek köszönhetően, nem volt szükség manuális SQL lekérdezések írására. A *Context* objektumon (*SzakemberKereso/Models/Context.cs*) végrehajtható LINQ utasítások kevesebb hibázási lehetőséget, és egy átláthatóbb forráskódot eredményeztek.

A *SzakemberKereso/Program.cs* fájl tartalmazza az Entity Framework Core és MySQL kapcsolat konfigurációját, az ASP.NET Identity és a Mapper leképezési beállításokat, a CORS szabályokat, valamint a FluentValidation és a két saját service regisztrációját. Az API végpontok kontroller metódusokként vannak definiálva, melyek közvetlenül az EF Core adatbázis-kontextuson dolgoznak.

5.1.2 Autentikáció és autorizáció

A RESTful architektúra egyik legnehezebben implementálható része a felhasználók hitelesítése, illetve az egyes erőforrások elérésének korlátozása. Az elkészült szoftver erre ASP.NET Identity alapú, süti hitelesítést alkalmaz. Sikeres bejelentkezés után a böngésző egy *AspNetCore.Identity.Application* nevű HttpOnly cookie-t kap, amelyet minden további kéréshez automatikusan elküld. A szerepkörök (szakember: „*Expert*”; átlag felhasználó: „*Generic*”) meghatározása után, az alkalmazás minden regisztrált felhasználót társít a megfelelő szereppel. Ezek a megfelelő attribútummal dekorált (*[Authorize(Roles = "...")]*), végpont metódusokon kerülnek érvényesítésre. Az anonim elérést igénylő endpointokon az *[AllowAnonymous]* attribútum szerepel.

5.1.3 Végpontok ismertetése

Az ASP.NET Core-ban a HTTP végpontokat kontrollerek definiálják. Ezek a *ControllerBase* szülő osztályból származnak, metódusaik az egyes végpontokat reprezentálják. A backend forráskájában, a *SzakemberKereso/Controllers/* almappában vannak definiálva. A kontroller szintű útvonalat a *[Route]* attribútum adja meg (pl. *[Route("api/[controller]")]*), az egyes metódusokat a HTTP method nevekkel végződő attribútumok jelölik (*[HttpGet]*, *[HttpPost]*, stb.), amelyek opcionálisan saját útvonal sablont is kaphatnak (pl.: *[HttpPost("{id}/offer")]*). A jogosultságkezelés az *[Authorize]* és *[AllowAnonymous]* attribútumokkal, szerepkör szinten az *[Authorize(Roles = "Expert")]* formával történik.

Az olvasáshoz fontos információ, hogy a végpontokat leíró táblázatokban, a végpontoknak elérési útja relatív, az azokat tartalmazó kontrollerhez képest. A teljes URL: [futó API IP cím és port]/[a kontroller nevét tartalmazó alcím, a '-' jel utáni rész]/[a táblázatban feltüntetett *Végpont* oszlop cellája]. Az *AuthController* estében például: *http://localhost:5272/api/register*.

5.1.3.1 *AuthController* - /api

A regisztrációs és bejelentkezési folyamatért felelős kontroller. Két regisztrációs útvonalat biztosít: az egyik általános felhasználót, a másik szakembert hoz létre. A bejelentkezés sikere esetén a szerver egy HttpOnly cookie-t állít ki, amelyet a böngésző automatikusan csatol minden további kéréshez.

Metódus	Végpont	Jogosultság	Leírás
POST	/register	Nyilvános	Általános felhasználó regisztrálása.
POST	/register/expert	Nyilvános	Szakember felhasználó regisztrálása.
POST	/login	Nyilvános	Bejelentkezés (cookie kiállítása).
POST	/logout	Bejelentkezett	Kijelentkezés (cookie törlése).

5-1. táblázat

5.1.3.2 *ExpertsController* - /api/experts

A szakemberprofil teljes körű kezeléséért felelős kontroller. Nyilvános végpontjain keresztül bárki böngészheti és szűrheti a szakemberek listáját, illetve megtekintheti egy-egy szakember profilját és elvégzett munkáit. A bejelentkezett szakember módosíthatja saját adatait, kezelheti szakterületeit és az azokhoz tartozó szolgáltatásait.

Metódus	Végpont	Jogosultság	Leírás
GET	/	Nyilvános	Szakemberek listázása (szűrőparaméterekkel: occupationId, minFixedPrice, maxFixedPrice, countyName, settlementName, allowUnitBased, unitName).
GET	/truncated	Nyilvános	Szakemberek egyszerűsített listája (id + teljes név).
GET	/ {id}	Nyilvános	A megadott id-val rendelkező szakember publikus profil adatai.
GET	/ {id}/jobs	Nyilvános	A megadott id-val rendelkező szakember, <i>Completed</i> státuszban lévő munkáit adja vissza.

GET	/me	Expert	A bejelentkezett szakember saját profilja.
PUT	/me	Expert	Profiladatok módosítása.
GET	/me/jobs	Expert	A bejelentkezett szakember összes munkamegbízása.
POST	/me/occupations	Expert	A bejelentkezett szakemberhez új szakterület hozzáadása (<i>?occupationId=[FEOR kód]</i>).
POST	/me/occupations/{occupationId}/services	Expert	A bejelentkezett szakember egy, már meglévő foglalkozásához, új szolgáltatás hozzáadása.
PUT	/me/occupations/{occupationId}/services/{serviceId}	Expert	A bejelentkezett szakember egy, már meglévő szolgáltatásának módosítása.

5-2. táblázat

5.1.3.3 JobsController - /api/jobs

A munkamegbízások életciklusát egy előző fejezetben ismertettük (4.3.7 jobs). Ennek kezeléséről, ez a kontroller felel. Minden végpont bejelentkezést igényel, az egyes átmenetek csak az érintett félnek engedélyezettek. A szerver minden kérésnél ellenőrzi, hogy a hívó valóban résztvevője-e az adott megbízásnak, és hogy az állapotátmenet az aktuális státuszból lehetséges-e.

Metódus	Végpont	Jogosultság	Leírás
GET	/id	Érintett fél	A munka lekérdezése.
POST	/	Generic	Új munkamegbízás létrehozása, <i>Pending</i> állapottal.
POST	/id/offer	Expert	Árajánlat és időpontjavaslat küldése. Állapotátmenet: <i>Pending</i> -ről <i>Offered</i> -re.
POST	/id/accept	Generici	Ajánlat elfogadása. Állapotátmenet: <i>Offered</i> -ről <i>Accpeted</i> -re.
POST	/id/complete	Expert	Munka elvégzettként jelölése. Állapot átmenet: <i>Accpeted</i> -ről <i>Completed</i> -re.
POST	/id/cancel	Bármelyik fél	Megbízás lemondása. Állapotátmenet: Bármelyik korábbi állapotról <i>Cancelled</i> -re. A munka <i>CacelledFromStatus</i> attribútuma tartalmazza az előző állapotot.
POST	/id/rate	Genric	Értékelés leadása. Kizárólag <i>Completed</i> állapotú munkák esetén.

5-3. táblázat

5.1.3.4 OccupationsController - /api/occupations

A FEOR-alapú foglalkozási katalógust kiszolgáló vezérlő. Egyetlen nyilvános végpontot tartalmaz, amely visszaadja az összes elérhető foglalkozást.

5.1.3.5 ServicesController - /api/services

A szolgáltatásokhoz kapcsolódó segédlekérdezéseket biztosító kontroller. Nem a CRUD-műveleteket kezeli (azok az *ExpertsController*-en belül találhatóak), hanem a keresési és szűrési felületek számára szükséges összesített adatokat adja vissza, mint a min/max árak vagy az adott szolgáltatás esetében elérhető egységárnevek.

Metódus	Végpont	Jogosultság	Leírás
GET	/pricingOptions	Nyilvános	Árszűrő adatok lekérdezése (<i>?occupationId=[FEOR kód]</i>): min/max ár, egységárnevek
GET	/truncated	Nyilvános	Új munkamegbízás létrehozása, <i>Pending</i> állapottal.
POST	/ {id} /offer	Expert	Árajánlat és időpontjavaslat küldése. Állapotátmenet: <i>Pending</i> -ről <i>Offered</i> -re.
POST	/ {id} /accept	Generici	Ajánlat elfogadása. Állapotátmenet: <i>Offered</i> -ről <i>Accpeted</i> -re.
POST	/ {id} /complete	Expert	Munka elvégzettként jelölése. Állapot átmenet: <i>Accpeted</i> -ről <i>Completed</i> -re.
POST	/ {id} /cancel	Bármelyik fél	Megbízás lemondása. Állapotátmenet: Bármelyik korábbi állapotról <i>Cancelled</i> -re. A munka <i>CacelledFromStatus</i> attribútuma tartalmazza az előző állapotot.
POST	/ {id} /rate	Genric	Értékelés leadása. Kizárólag <i>Completed</i> állapotú munkák esetén.

5-4. táblázat

5.1.3.6 SettlementsController – api/settlements

A település adatok lekéréséhez szükséges kontroller. Olyan végpontot nem definiáltunk, ami az összes eltárolt települést lekérdezi, mivel az túlságosan nagy mennyiségű adatátviteléhez vezetne. A frontendet úgy alakítottuk ki, hogy ne legyen egyszerre szükség az összes településre. Mindenhol kétlépcsős megoldást alkalmaztunk: először lekérdezzük a vármegyéket, majd a kiválasztotthoz tartozó településeket.

Metódus	Végpont	Jogosultság	Leírás
GET	/counties	Nyilvános	Visszaadja azon vármegyék neveit, melyekben legalább 1 szakember működik.
GET	/byCounty	Nyilvános	Visszaadja azon, a megadott vármegyén belül elhelyezkedő településeket, melyekben legalább 1 szakember működik.

GET	/allCounties	Nyilvános	Visszaadja az összes vármegye nevét.
GET	/allByCounties	Nyilvános	Visszaadja az összes települést a meghatározott vármegyében.

5-5. táblázat

5.1.3.7 UsersController - /api/users

A bejelentkezett felhasználó saját adatait kezelő kontroller. Lehetővé teszi a név és telefonszám-módosítást, visszaadja az aktív szerepköröket, valamint lekérhetőek rajta keresztül a felhasználó által indított munkamegbízások.

Metódus	Végpont	Jogosultság	Leírás
GET	/me	Bejelentkezett	Saját felhasználói adatok és szerepkörök lekérése.
PUT	/	Bejelentkezett	Felhasználói adatok módosítása.
GET	/me/jobs	Bejelentkezett	A bejelentkezett felhasználó által indított megbízások lekérdezése.

5-6. táblázat

5.1.4 DTO-k, Mapper és FluentValidation

Az API nem közvetlenül az entitásosztályokat adja vissza a kliensnek, hanem adatátviteli objektumokat (*Data Transfer Object*). A DTO-k (*SzakemberKereso/DTOs*) két irányban működnek: a bemeneti DTO-k (*Input...*, *Create...*, *Update...*) a kienstől érkező adatokat modellezik, míg a kimeneti DTO-k (*Output...*) az API válaszáának alakját határozzák meg.

Az entítások és kimeneti DTO-k közötti átalakítást több helyen a *Mapper* könyvtár végzi. Ez segít a kód átláthatóságán, hiszen egy metódus (*ProjectToType<T>()*), intelligensen, sok esetben külön konfigurálás nélkül megcsinálja a konverziót. Képes beépülni a LINQ kifejezések láncolatába, melynek segítségével az EF Core által generált SQL lekérdezéseket optimalizálja. Néhány entitás – DTO párosnak a konvertálását a csomag nem tudta megoldani. Ezek az egyedi leképezési szabályok a *ConfigureMapper()* (*SzakemberKereso/Program.cs*) metódusában kerülnek regisztrálásra. Vannak végpont metódusok, ahol manuális átalakítás van használatban.

A beérkező DTO-k validálása, részben az érintett végpontok törzsében, részben pedig a FluentValidation csomag alkalmazásával történik. A keretrendszer alpból elveti a rosszul formált kéréseket, de előfordulhatnak olyan hibák, amelyeket nem képes kezelni. Például a *PricingDto* esetében, *fix árazási* típus esetén nem, de *fix- és egységáras* esetén

szükséges a mértékegység megnevezése. Ez, illetve hasonló a szabályok a DTO osztályok állományában vannak definiálva, majd *dependency injection* segítségével, az igénylő kontrollerekben kerülnek ellenőrzésre. A csomag alpból támogatja, a különféle validációk egymásba ágyazását. Így, ha valamelyik másik átmeneti objektumban szükség van *PricingDto-ra*, az erre meghatározott szabályok ismét ellenőrzésre kerülnek. A FluentValidation csomag sokat segít a kód tisztaságán és átláthatóságán.

5.2 Frontend - Next.js alkalmazás

A felhasználóknak számára, az alkalmazás által nyújtott lehetőségeket, egy modern kinézetű, Next.js 16 alapú webes kliens teszi elérhetővé. A reszponzív szerkezetének köszönhetően, telefonon és asztali számítógépen is biztosítja a megfelelő felhasználói élményt.

5.2.1 Next.js használata

A React alapú Next.js, számunkra legmeghatározóbb előnye, hogy a forráskód könyvtárstruktúrája határozza meg a webes platformon elérhető útvonalakat. Ebből kifolyólag nem volt szükség külön router csomag telepítésére, a keretrendszer alpból biztosított minden ehhez szükséges funkciót, komponenst. Ez sokat segít a frontend projekt átláthatóságán, hiszen minden oldalt az *src/app/* mappán belül elhelyezett *page.js* fájlok határoznak meg. Ez lehetőséget adott arra, hogy az egyes oldalakhoz szükséges komponenseket, tisztán és egyszerűen, egy külön alkönyvtárban (*src/components/*) definiáljuk. A zárójelbe tett mappák (*(public)*, *(user_public)*) úgynevezett *route group-ok*, amelyek segítségével közös *layout* alkalmazható az alattuk lévő oldalakra anélkül, hogy a mappa neve megjelenjen az URL-ben.

Az elérhető útvonalakat az alábbi táblázta foglalja össze:

URL	Leírás
/	Főoldal
/search	Az alkalmazás kereső funkcióját egységbe záró oldal.
/auth/login	Bejelentkezési űrlap.
/auth/register	Regisztrációs űrlap. Általános és szakember felhasználók egy helyen regisztrálhatnak.
/dashboard	Átírányítás a <i>/dashboard/profile</i> oldalra.
/dashboard/jobs	Mindkét felhasználó típus számára elérhető. Lehetővé teszi a profilhoz tartozó munkák megtekintését, és szerkesztését.
/dashboard/occupations	Csak a szakember felhasználók számára érhető el. Lehetővé ad foglalkozások meghatározására.

/dashboard/services	Csak a szakember felhasználók számára érhető el. Lehetőséget ad foglalkozások hozzáadására és szerkesztésére.
/dashboard/profile	Mindkét felhasználó számára elérhető. Profiladatok szerkesztésére ad lehetőséget.
/experts/[slug]	Egy meghatározott id-val rendelkező szakember publikus profiljának összefoglalója. Bejelentkezés nélkül is elérhető.
/experts/[slug]/services	Egy meghatározott id-val rendelkező szakember szolgáltatásait listázza. Bejelentkezés nélkül is elérhető.
/experts/[slug]/jobs	Egy meghatározott id-val rendelkező szakember már teljesített munkáit listázza. Bejelentkezés nélkül is elérhető.

5-7. táblázat

A `/dashboard/*` útvonalakat a `src/proxy.js` middleware védi: az autentikációs süti hiányában a felhasználót a `/auth/login?from=<eredeti-url>` oldalra irányítja át. A dashboard oldalak betöltődése előtt, ezeknek a közös layout komponense (`/dashboard/layout.js`) alaposabban ellenőrzi, a felhasználói munkamenet érvényességét.

Hasznát vettük a `Rewrites` funkciónak is. A frontend az összes `/api/*` kérést automatikusan a backend API (`http://localhost:5272/api/`) végpontokra irányítja át, ami a `/NextAppSzakemberKereso/next.config.mjs` állományban van meghatározva. Így a kliens nem közvetlenül a backend-del, hanem a Next.js serverrel kommunikál. Ez a kód tisztasága érdekében is hasznos, hiszen nem szükséges minden fetch kérésnél meghatározni az API alapútvonalát.

5.2.2 Server Side Rendering

A Next.js az App Router architektúrán keresztül alapértelmezetten szerveroldali renderelést (*Server-Side Rendering*) alkalmaz. Ez azt jelenti, hogy az adott komponensek HTML tartalma a szerveren kerül előállításra, és a böngésző már egy teljes, megjelenítésre kész DOM node-ot kap, nem csak egy üres vázat, amelyet scriptek töltenek fel adatokkal. Ez felgyorsítja egy-egy oldal betöltődését, hiszen a kliensnek nem kell megvárni az összes kód letöltődését és végrehajtását. Voltak komponensek, amelyeken ezt tudtuk alkalmazni, de sok esetben ez nem volt lehetséges, főként azért, mert a szerver oldali komponensek nem támogatják a *React hook*-ok használatát. Így, ha szerettünk volna például *useState*-et használni, akkor a komponenst tartalmazó fájlt meg kellett jelölni a `'use client'` direktívával, ami arra az állományra kapcsolja a *server side renderinget*.

5.2.3 Material UI

A keretrendszer mellett szükségünk volt egy komponenskönyvtárra is, ami lehetőséget ad egy szép és modern kinézetű weboldal megvalósításához. A Google által kialakított *Material Design* séma és szabályrendszer kifejezetten tetszett nekünk. Így esett a választás a Material UI (MUI) komponens könyvtárra, amely az utóbbi által felállított standardokat implementálja. A kiválasztott library több száz kidolgozott React-komponenst tartalmaz, például gombokat, párbeszédablakokat, léptetőket, dátumválasztókat és még sok mást.

Hasznos funkciója volt az, hogy az alkalmazás vizuális megjelenését egyetlen, központi témafájlban (*src/app/theme.js*) tudtuk egységesen konfigurálni. Itt került beállításra a színpaletta, illetve lehetőséget adott a beépített MUI komponensek alapértelmezett viselkedésének és kinézetének változtatására. Ez biztosította azt, hogy az összes oldal megjelenése egységes maradjon anélkül, hogy minden komponensnél külön kellene stílusokat megadni. A *MUI X Date Pickers kiegészítő* csomag a megbízások időpontegyeztetési felületéhez nyújtott komplex, mégis könnyen integrálható dátum- és idő kiválasztó beviteli mezőket.

5.2.4 Egyedi React hook-ok és SWR

Sok komponensnek szüksége van a felhasználói adatokra, amelyek az *api/experts/me*, illetve az *api/users/me* végpontokon érhetők el. Minden egyes helyen külön fetch kéréseket írni, majd az ahhoz tartozó állapotokat (error, succes, loading) implementálni, sok boilerplate kódot adott volna hozzá a forráskódhoz. Ennek a problémának a megoldására a Next.js fejlesztői által készített és fenntartott SWR (Stale While Revalidate) könyvtárat használtuk. Ez a csomag főként optimalizálási célból lett megalkotva. Enkapszulál egy fetch kérést, és ameddig az folyamatban van, addig a gyorsítótárazott, régebbi adatokat adja vissza. A kérés teljesülésekor összehasonlítja az előző választ az újjal, és ha észlel változást, újrendererli a használó komponenst. Kezeli a betöltési állapotot, a hibát és az újra lekérést is, így ezeket nem kellett `useEffect` és `useState` kombinációjával manuálisan megvalósítani. Számunkra inkább ez a funkciója lett meghatározó. Ennek a segítségével tudtunk két, újrahasznosítható egyedi React hookot készíteni:

- useUser(): az `/api/users/me` végpont adatait kezeli. Az összes oldal, amelynek ismernie kell a bejelentkezett felhasználó adatait (névsor, szerepkör), ezt a hook-ot használja.
- useExpert(): az `/api/experts/me` végpont adatait kezeli, azonos szerkezettel. Kizárólag az *Expert* szerepkörű felhasználók Dashboard oldalain kerül meghívásra.

6 Működés műszaki feltételei

6.1 Szükséges szoftverek

Szoftver	Verzió	Letöltés
.NET SDK	8.0	https://dotnet.microsoft.com/en-us/download
Node.js	22.21.0	https://nodejs.org/en
MySQL	8.0	Lásd: 6.3 Adatbázis előkészítése

6-1. táblázat

6.2 A projekt mappaszerkezete

A repository mappaszerkezete:

- BackendSzakemberKereso/: (a backend projekt gyökere)
 - SzakemberKereso (fő backend projekt)
 - SzakemberKereso.Tests (backend tesztek)
- NextAppSzakemberKereso/: (a frontend projekt gyökere)
- database/
 - docker/
 - docker-compose.yml
 - vrdb_partial_dump.sql (fő dump)
 - table_settlements.sql (összes magyarországi település)
 - table_feor.sql (összes FEOR foglalkozás)

6.3 Adatbázis előkészítése

6.3.1 Adatbázis dump fájlok

A `/database/vrdb_partial_dump.sql` tartalmazza a teljes adatbázissémát, illetve a táblák feltöltéséhez szükséges adatokat. Ez az egyetlen fájl, amely szükséges az alkalmazás futtatásához.

A `/database/table_settlements.sql` és a `/database/table_feor.sql` önálló referenciatáblák, amelyek a már létező `occupatoins` és `settlements` teljese feltöltéséhez szükséges SQL utasításokat tartalmazzák. Ezek méretük miatt nem kerültek bele a fő

dumpba. Importálásuk opcionális: a fő `/database/vrdb_partial_dump.sql` egy kisebb, de funkcionálisan teljes adatkészletet tartalmaz belőlük.

Az *occuaptions* tábla teljese feltöltéséhez, a FEOR kódokat és az azt jelölő foglalkozások megnevezését, a Központi Statisztikai Hivatal weboldaláról került letöltésre és átalakításra. (forrás: <https://www.ksh.hu/docs/szolgalattasok/hun/feor08/feorlista.html>)

A *settlements* tábla feltöltésére a <https://github.com/ferenci-tamas/IrszHnk/> publikus GitHub repositoriban talált adatokat használtuk.

A *BackendSzakemberKereso/SzakemberKereso/appsettings.json* fájlban a kapcsolati adatok szükség esetén módosíthatók. (jelenlegi beállítás: `server=localhost;port=3306;uid=root;pwd=;database=vrdb`)

6.3.2 Futtatás – Dockert használatával

A `/database/docker` almappában a *docker compose up -d* parancs kiadását követően letöltésre kerülnek a szükséges image fájlok, és egy kis idő elteltével a MySQL adatbázis elérhetővé válik a `http://localhost:3306` címen. (felhasználó: `root`, jelszó: `nincs`). A phpMyAdmin kezelőfelület a <http://localhost:8080/> címen megtekinthető.

A Docker Compose konfiguráció automatikusan elindít egy MySQL 8.0 és egy phpMyAdmin konténert. A `/database/vrdb_partial_dump.sql/` az első indításkor automatikusan betöltődik, külön import nem szükséges. A phpMyAdmin nem kötelező, de egy light-weight, egyszerűen használható felületet ad az adatbázis megtekintésére, kezelésére.

Referenciafájlok opcionális importálása:

- 1 `docker exec -i mysql_database mysql -u root vrdb < ../table_settlements.sql`
- 2 `docker exec -i mysql_database mysql -u root vrdb < ../table_feor.sql`

6.3.3 Futtatás – Natív MySQL telepítés

Amennyiben MySQL 8.0 már telepítve van a futtatási környezetben, az adatbázis a következő paranccsal hozható létre és tölthető fel:

- 1 `mysql -u root -e "CREATE DATABASE IF NOT EXISTS vrdb;"`
- 2 `mysql -u root vrdb < database/vrdb_partial_dump.sql`

Referenciafájlok opcionális importálása:

```
1 mysql -u root vrdp < database/table_settlements.sql
2 mysql -u root vrdp < database/table_feor.sql
```

6.3.4 Elérhető települések (fő dump)

A `/database/vrdp_partial_dump.sql` importálása után az alábbi 15 település érhető el az alkalmazásban. Munkakérés létrehozásakor a helyszín megadásánál ezek az irányítószámok és településnevek használhatók. Lásd: *9.3 Fő dump települései*.

6.3.5 Előre meghatározott felhasználói fiókok

Lásd: *9.4 Felhasználók listája*

6.4 Futtatás – Production környezet

Ez az ajánlott üzemmód a vizsga során. Ebben az üzemmódban a Swagger felület és a tesztadat-generátor nem aktív; az adatbázis-séma és az adatok a dump-fájlból kerülnek előkészítésre. Ebből kifolyólag fontos, hogy az adatbázis működőképes állapotban legyen. (Lásd: *6.3 Adatbázis előkészítése*)

6.4.1 Backend indítása

```
1 cd BackendSzakemberKereso/SzakemberKereso
2 dotnet run --launch-profile production
```

A backend a `http://localhost:5272` címen fog elindulni.

6.4.2 Frontend indítása

```
1 cd NextAppSzakemberKereso
2 npm install
3 npm run build
4 npm start
```

A frontend a `http://localhost:3000` címen, illetve lokális hálózaton is elérhető lesz. Fontos, hogy a backend projekt futó állapotban (lásd: *6.4.1 Backend indítása*), és a `http://localhost:5272` címen legyen elérhető. Utóbbin eszközölt változtatás esetén, a `NextAppSzakemberKereso/next.config.mjs` állományban szükséges ehhez igazítani a `destination` mezőben meghatározott URL-t.

6.5 Futtatás – Fejlesztői környezet

Ebben a bekezdésben leírjuk az általunk használt fejlesztői konfiguráció elindítását. Fontos megjegyezni, hogy ebben az üzemmódban az alkalmazás automatikusan futtatja az adatbázis migrációkat és indításkor feltölti azt tesztadatokkal (*SzakemberKerso/Utils/DummyDataGenerator.cs*), így külön dump-visszatöltés nem szükséges. Elérhetővé válik a Swagger UI is.

6.5.1 Adatbázis indítása

```
1 cd database/docker
2 docker compose up -d
```

6.5.2 Backend indítása

```
1 cd BackendSzakemberKereso/SzakemberKereso
2 dotnet run --launch-profile http
```

A backend a *http://localhost:5272* címen indul el. Az első indításkor az EF Core automatikusan lefuttatja a migrációkat és legenerálja a tesztadatokat. A Swagger UI a *http://localhost:5272/swagger* címen érhető el.

6.5.3 Frontend indítása

```
1 cd NextAppSzakemberKereso
2 npm install
3 npm run dev
```

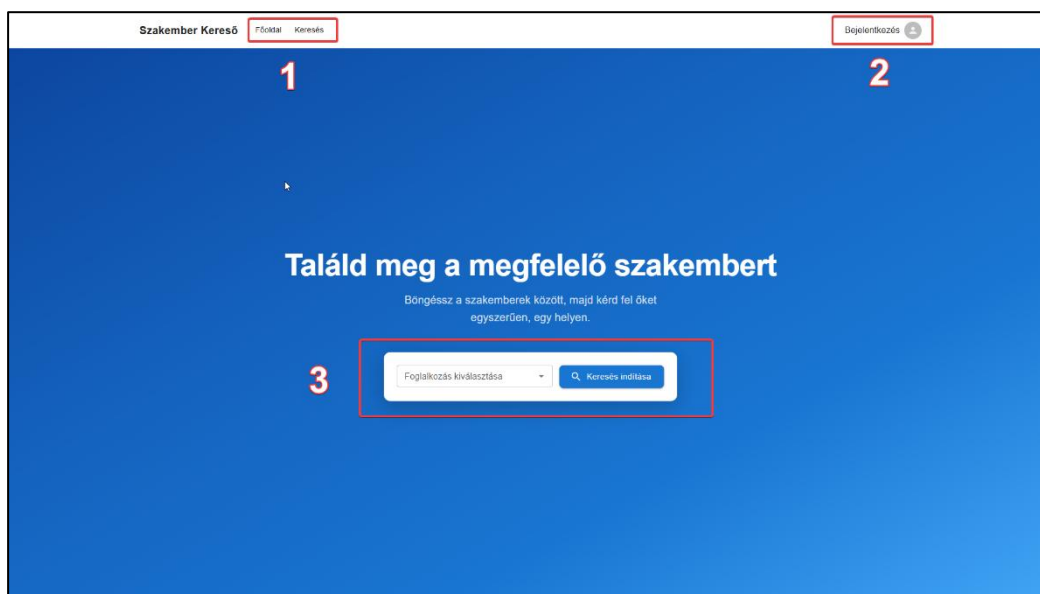
A frontend a *http://localhost:3000* címen lesz elérhető.

7 Az alkalmazás működése

7.1 Kezdő oldal

A webalkalmazás *landing page*-e.

7.1.1 Az oldal felépítése

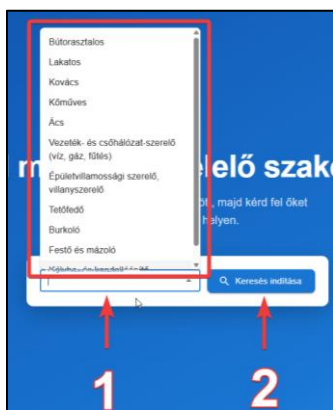


7-1. sz. kép

7-1.sz. kép	Leírás
1	Navigációs menüpontok, melyekkel az adott oldalra (<i>Kezdő oldal</i>) és a <i>Kereső</i> oldalra tudja magát navigálni a felhasználó.
2	Bejelentkezés menüpont. A felhasználó a <i>Bejelentkezés</i> oldalra irányítódik.
3	A felhasználó átnavigál a <i>Kereső oldalra</i>

7-1. táblázat

7.1.2 Keresés indítása



7-2. sz. kép

Beviteli mezőbe kattintással egy felugró listából (7-1. sz. kép 1) van lehetőségünk kiválasztani a keresni kívánt foglalkozást. Szöveges bevittel is lehet a foglalkozások közt keresni. Kizárólag a listában szereplő foglalkozásokból tudunk választani, egyszerre egyet. „Keresés indítása” feliratú gombbal (7-2. sz. kép 2) indíthatjuk el a keresést a foglalkozás kiválasztása után. A felhasználó átirányítódik a *Kereső oldalra*.

Kitöltetlen vagy helytelen kitöltés esetén a keresési mező üresen marad, minden egyes szakember megjelenítésre kerül.

7.2 Bejelentkezés, regisztráció

7.2.1 Bejelentkezés

Bejelentkezés menüpontra (7-1. sz. kép 2) kattintás után a felhasználó be tud jelentkezni a már korábban létrehozott fiókjába. Az autentikációhoz szükség van az email (7-3. sz. kép 1) és a jelszó (7-3. sz. kép 2) mezők kitöltésére. Ezek hiánya, vagy hibás

7-3. sz. kép

megadása esetén a bejelentkezés sikertelen lesz és figyelmeztető üzenet jelenik meg. Helyes email jelszó párossal a felhasználó a „Bejelentkezés” gomb lenyomása után sikeresen bejelentkezik. A korábbi *Bejelentkezés menüpont* (7-1. sz. kép 2) helyén a saját neve jelenik meg. Ha a felhasználó esetleg még nem regisztrált az oldalra, akkor ezt a „Regisztrálj” hivatkozásra (7-3. sz. kép 3) kattintva megetheti. (A létező felhasználó bejelentkezési adatai a 9.4 *Felhasználók listája* szekcióban található)

megadása esetén a bejelentkezés sikertelen lesz és figyelmeztető üzenet jelenik meg. Helyes email jelszó párossal a felhasználó a „Bejelentkezés” gomb lenyomása után sikeresen bejelentkezik. A korábbi *Bejelentkezés menüpont* (7-1. sz. kép 2) helyén a saját neve jelenik meg.

7.2.2 Regisztráció

7-4. sz. kép

Lehetősége van a felhasználóknak saját profilt regisztrálni, mellyel később bejelentkeznek a felületre. A két típusú felhasználó regisztrációja eltérő. Ezért egy feltűnő, kapcsoló stílusú gombbal (7-4. sz. kép 1) tudunk választani, hogy általános felhasználóként vagy szakemberként regisztrálunk.

Választott lehetőségtől függően más adatokat kell megadni a felhasználónak. A kötelező mezőket (7-4. sz. kép 2), mindkét esetben szükséges kitölteni a regisztrációhoz.

Ezen kívül látható még a „Regisztráció” gomb (7-4. sz. kép 3), illetve a „Bejelentkezés” hivatkozás (7-4. sz. kép 4), mellyel a felhasználónak lehetősége van visszatérni a bejelentkezési oldalra.

7.2.3 Regisztráció szakemberként

7-5. sz. kép

Szakember-felhasználóként történő regisztráció esetén szükséges megadni további adatokat (7-5. sz. kép 1). A vállalkozás székhelyének vármegye és település adatait csak a felugró lista egyik elemének kiválasztásával lehet meghatározni. Települést addig nem enged a felület kiválasztani, amíg a vármegye nem került megadásra. Kizárólag a kiválasztott vármegye települési jelennek meg.

7.2.4 Regisztráció általános felhasználóként

7-6. sz. kép

Általános-felhasználói profil regisztrációjához csak a kötelező, csillaggal jelölt adatokat kell megadni.

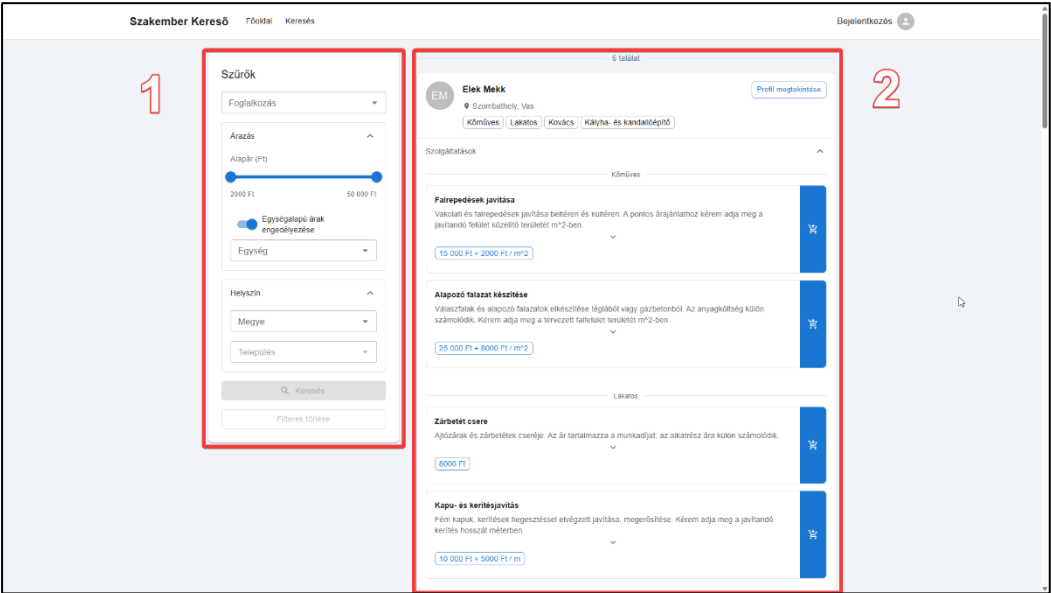
A 7-6. sz. képhez tartozó jelölések és azok magyarázata:

1. Jelszót a két kijelölt mezőben kell megadni, mindkét mező tartalmának egyeznie kell.
2. Abban az esetben, ha a megadott jelszó nem felel meg a kritériumoknak, a rendszer figyelmeztető üzenettel jelez.
3. Az alkalmazás pontosan felsorolja azokat a szabályokat, melyeknek a megadott jelszó nem tesz eleget. Egyes esetekben, aktív számláló jelzi a tévedés mértékét.

7.3 Kereső oldal

Bejelentkezés nélkül is elérhető. Szakemberek és szolgáltatások keresésére, szűrésére ad lehetőséget.

7.3.1 Az oldal felépítése



7-7. sz. kép

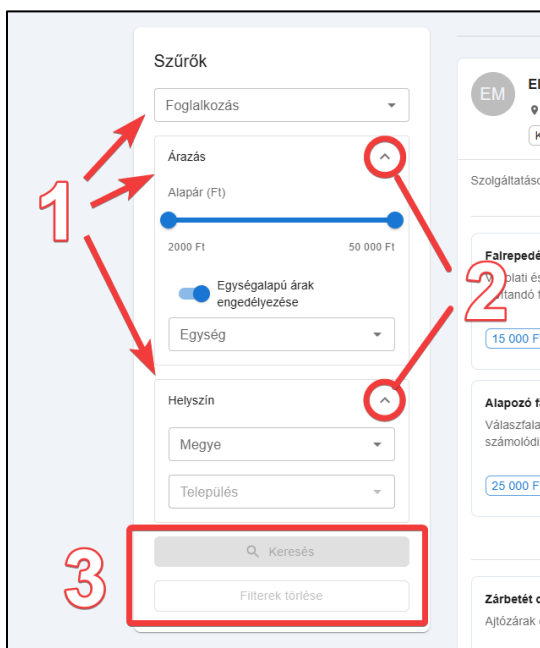
Keresés oldalt a főoldal tetején található navigációs sávból (7-1. sz. kép 1), valamint a „Keresés indítása” gombra (7-2. sz. kép 2) kattintás után lehet elérni.

7-7. sz. kép	Leírás
1	Szűrő panel, ahol a regisztrált szakemberekre, illetve az általuk nyújtott szolgáltatásokra állíthatunk keresési feltételeket.
2	Keresési találatok, melyek a megadott szűrési feltételeknek megfelelő szakembereket, illetve szolgáltatásokat listázza.

7-2. táblázat

7.3.2 Szűrő panel

Szakemberek között lehetőség van a szűrésre. Számunkra kívánt adatok megadása után megtalálhatjuk a megfelelő mesterembert. A felhasználónak lehetősége van foglalkozás, ár és helység szerint szűrésre.



7-8. sz. kép

A 7-8. sz. képhez tartozó jelölések és azok magyarázata:

1. A három szűrési opció.
2. Nyílak segítségével az egyes szűrési paraméterek összezárása és kinyitása van lehetőség, az átláthatóbb keresés érdekében.
3. „Keresés” gombra kattintva megjelennek a megadott feltételeknek megfelelő szakemberek, és azok szolgáltatásai. A „Filterek törlése” gombra kattintva, a Szűrés panel alaphelyzetbe kerül.

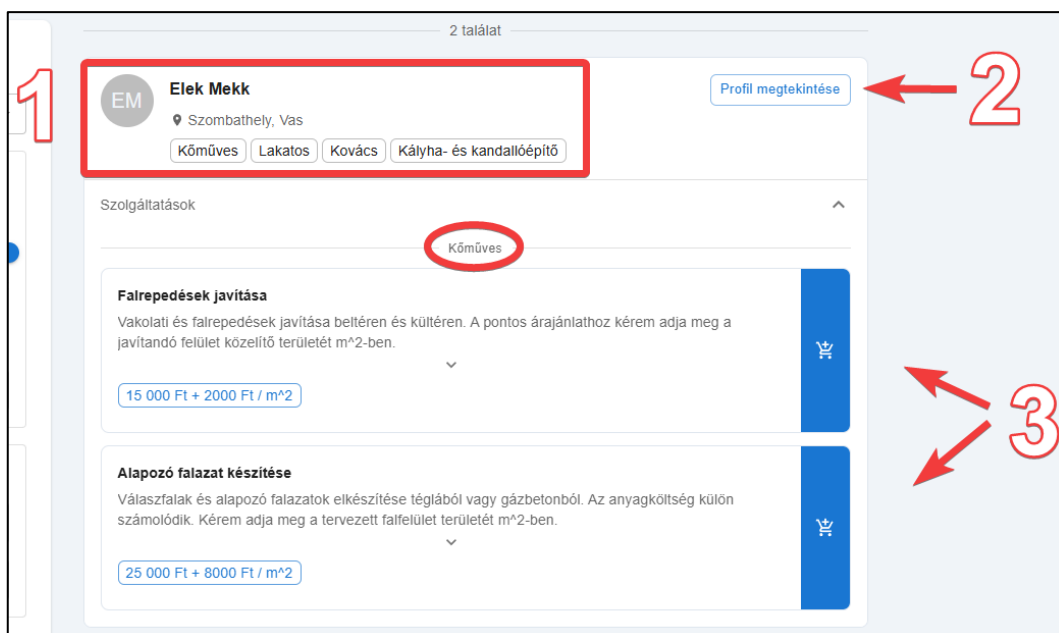
A szakterületet a *Szűrési panel* tetején lévő legördülő listából lehet kiválasztani. Abban az esetben, ha a *Kezdő oldalon* már kiválasztott a felhasználó egy foglalkozást, akkor ez a mező már kitöltött.

Az árazási intervallumot a csúszka segítségével lehet beállítani. A minimum és maximum árat (melyek a csúszka bal és jobb szélén vannak feltüntetve) illetve a választható egységeket, a meghatározott szolgáltatás befolyásolja. Ez az opció nem a szakemberekre, hanem az azok által kínált szolgáltatásokra vonatkozik. Nem jelennek meg azok a kínálatok, amelyek ára a megadott intervallumon kívül helyezkednek el, vagy amelyek nem a kiválasztott mértékegység alapján vannak díjazva. Lehetőségünk van az egység alapon árazott szolgáltatások teljes kikapcsolására.

A *Regisztrációnál* látottakhoz hasonlóan, először a vármegyét, majd a helységet kell kiválasztani a „Helyszín” opció alatt. Itt csak azok a vármegyék, illetve települések kerülnek feltüntetésre, amelyekhez tartozik legalább egy regisztrált felhasználó.

7.3.3 Találatok panel

Szűrési paramétereknek megfelelő szakemberek, és a hozzájuk tartozó szolgáltatások az oldalon, egy oszlopba rendezve jelennek meg.

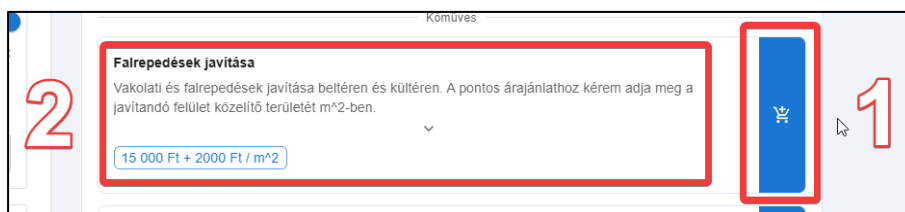


7-9. sz. kép

7-9. sz. kép	Leírás
1	Szakember információi: név, helyszín, foglalkozások.
2	A szakember publikus profiljára irányítja át az oldal használóját. Bejelentkezés nem szükséges.
3	Foglalkozásonként csoportosítva jelennek meg a szakember által kínált szolgáltatások. Kiválasztott szolgáltatás esetén, csak az ahhoz tartozó kínálatok jelennek meg.

7-3. táblázat

A szolgáltatásokat a rendszer kártyák formájában jeleníti meg, melyek további információkat és lehetőségeket kínálnak a kereső felhasználónak.



7-10. sz. kép

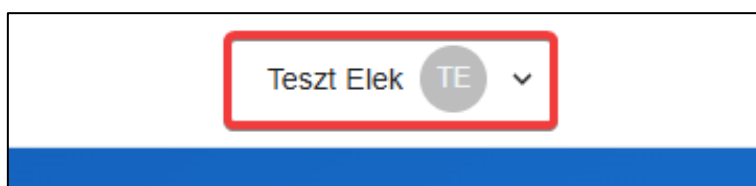
7-10. sz. kép	Leírás
1	Amennyiben a felhasználó be van jelentkezve, a gombra kattintás átirányítja, a profiljához tartozó kezelőfelületre, azon belül is a Munkák oldalra. Itt egyből megkezdheti a kérvényezési folyamatot.
2	Szolgáltatás információi: megnevezés, leírás és árazás.

7-4. táblázat

7.3.4 URL

Fontos megemlíteni, hogy a megadott szűrési feltételek az URL-ben is megjelennek. Ez azért előnyös, hiszen így a felhasználók könnyen megoszthatják egymással az általuk összerakott keresési beállításokat. Előnye még, hogy lehetőséget biztosít a korábbi keresések visszahozására, a böngésző navigációs gombjainak használatával. Illetve, az oldalt a keresési beállításokkal együtt lehet a webböngésző könyvjelzői közé menteni, ezért nincs szükség ugyanazon paraméterek újbóli beállítására.

7.4 Navigációs menü



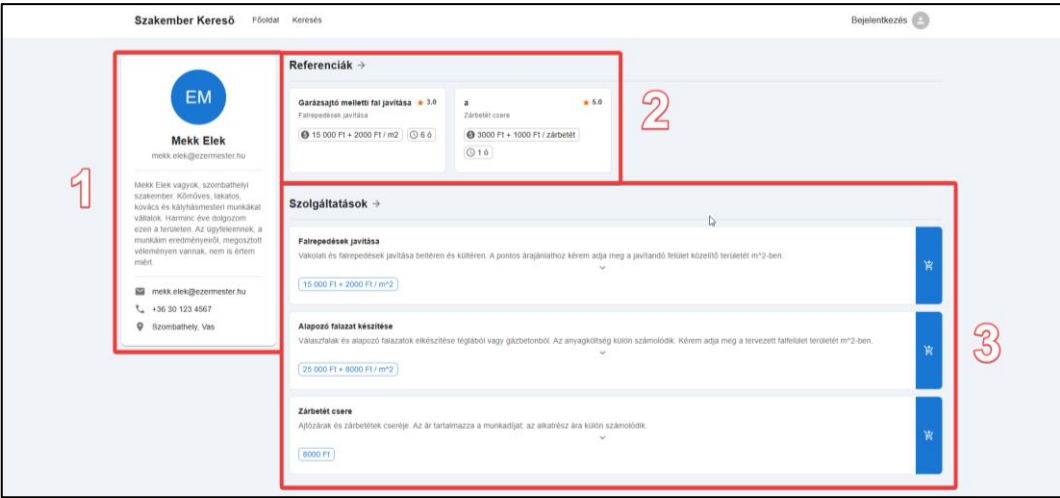
7-11. sz. kép

A 7-11. sz. képen bekeretezett komponenssel már a *Kezdő oldalon* találkoztunk (7-1. sz. kép 2). Ott azonban még „Bejelentkezés” szöveg volt látható, a felhasználó neve helyett. Ez már sejteti ennek az elemnek az egyik feladatát: jelzi, hogy az adott felhasználó rendelkezik érvényes munkamenettel vagy sem. Másik feladta, kattintásra megjeleníteni a különféle navigációs lehetőségeket. Ezek a bejelentkezett felhasználó típustól, illetve az aktuális útvonaltól függően eltérőek, de mivel egy helyen, minden oldal jobb felső sarkában helyezkedik el, az alkalmazás használója navigációs célokból ösztönösen ezt a komponenszt fogja használni. Harmadik feladata: biztosítja a kijelentkezési lehetőséget.

A navigációs menü lehetséges megjelenéseinek bemutatása (lásd: 7-5. sz. táblázat)

7.5 Szakember publikus profilja

7.5.1 Az oldal felépítése



7-12. sz. kép

A *Kereső* oldalon, a *Találatok* panelen belül elhelyezett *Profil megtekintése* (7-9. sz. kép 2) gombra kattintva lehetőség adódik a szakember részletesebb adatainak megtekintésére.

7-12. sz. kép	Leírás
1	Szakember adatai és rövid bemutatkozása.
2	Néhány korábbi, már befejezett munka adatainak megjelenítése (maximum 3 darab).
3	A szakember 3 szolgáltatásának feltüntetése.

7-5. táblázat

A „Szolgáltatások” felírra kattintva, a felület megjeleníti a szakember minden kínálatát, foglalkozás alapján csoportosítva. A szolgáltatásokat mindhárom helyen (*Kereső oldal*, *experts/{id}*, *experts/{id}/services*) ugyanazzal a kártyával vannak megjelenítve, így biztosítva az egyértelmű, konzisztens felhasználói élményt.

A „Referenciák” felírra kattintva, megjelenik a szakember által elvégzett, befejezett státuszú munkák. Ezeknek értékelése is látható, amennyiben a megbízó azt meghatározta.

7.6 Dashboard

Minden regisztrált és bejelentkezett felhasználó, a profil adatok frissítését, illetve a munkafolyamat kezelését az irányítópulton, azaz a *Dashboard-on* teheti meg. Érvényes munkamenet nélkül, nem lehet elérni a felületet.

7.6.1 Az oldal felépítése

Minden *Dashboard* oldal bal szélén vannak felsorolva az adott felhasználó által elérhető útvonalak. A navigációs listaelemekre kattintással térhetünk át egy új oldalra, ami attól jobbra fog megjelenni.

7.6.2 Profil beállítások oldal

Személyes adatok

Vezetéknév: Mekk

Keresztnév: Elek

Telefonszám:

Szakember adatok

Bemutatószöveg: Mekk Elek vagyok, szombathelyi szakember. Kőműves, lakatos, kovács és kályhásmeisteri munkákat válllok. Harminc éve dolgozom ezen a területen. Az ügyfeleimnek, a munkám eredményeiről, megosztott véleményem vannak, nem is értem miért.

Vállalkozás telefonszáma: +36 30 123 4567

Vállalkozás emailje: mekk.elek@ezermester.hu

Vármegye: Vas

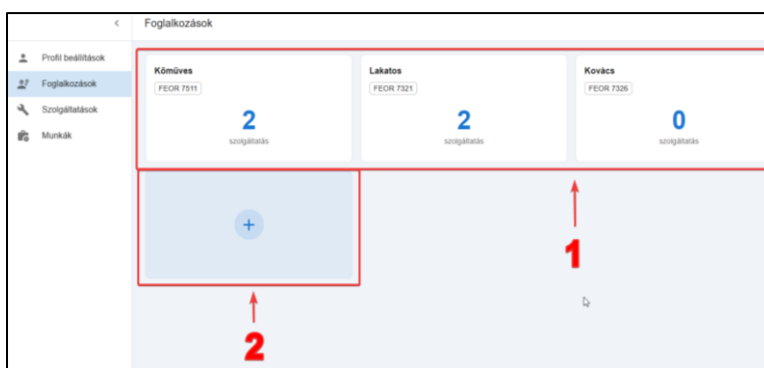
Település: Szombathely (9700)

Mentés

7-13. sz. kép

Itt láthatók, illetve módosíthatók az adott profilhoz tartozó adatok (7-13. sz. kép 2). Ez az oldal, az általános és a szakember felhasználók számára is elérhető. Az utóbbi esetben azonban kiegészül az adott profil típus által támogatott extra attribútumok beállítására lehetőséget adó mezőkkel („*Szakember adatok*”).

7.6.3 Foglalkozások oldal



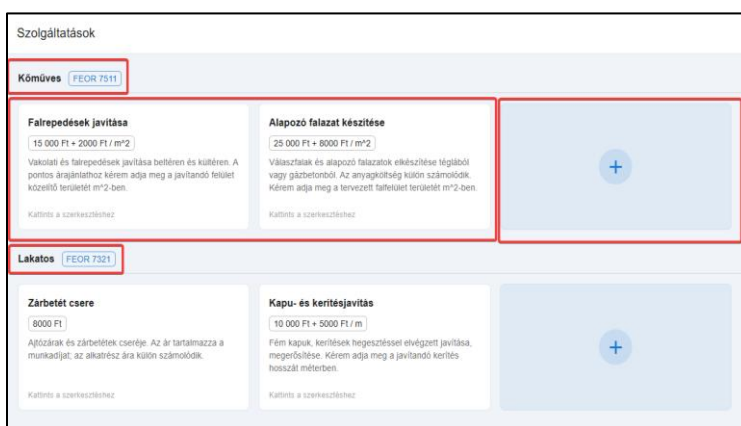
7-14. sz. kép

Kizárólag egy szakember felhasználó által érhető el. Az oldal a mesterember korábban hozzáadott foglalkozásait, illetve azon belüli szolgáltatásainak számát jeleníti meg, kártyák formájában (7-14. sz. kép

1). Lehetőséget ad, új foglalkozások felvételére (7-14. sz. kép 2).

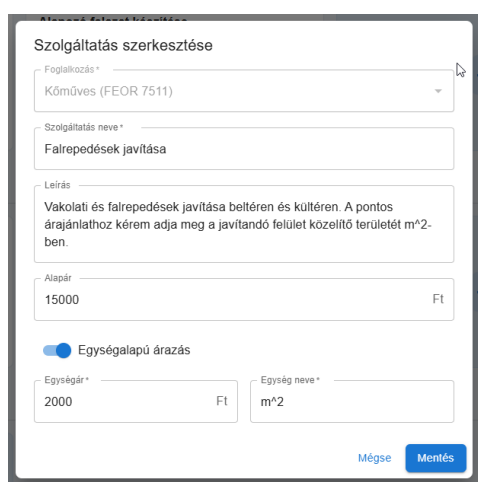
7.6.4 Szolgáltatások oldal

Az oldal a szakemberek számára elérhető, azoknak korábban létrehozott



7-15. sz. kép

szolgáltatásait mutatja. Ezek, foglalkozások szerint csoportosítva jelennek meg. Lehetőség adódik, új szolgáltatás készítésére, illetve a már meglévők módosítására, a kártyára történő kattintással.



7-16. sz. kép

A módosításra, illetve az új kínálat hozzáadására felépített dialógus ablak nagy részében megegyező. A különbséget az adja, hogy egy szolgáltatás hozzáadását követően a szakembernek nincs lehetősége az ahhoz tartozó foglalkozás megváltoztatására (7-16. sz. kép: *Foglalkozás mező inaktív*).

7.6.5 Munkák oldal

A szakember és az általános felhasználók számára legfontosabb *Dashboard* oldal. Utóbbi esetén feltüntetésre kerülnek a kérvényezett munkák. Az általános felhasználók pedig láthatják az általuk indítványozott kérelmeket.

7-17. sz. kép

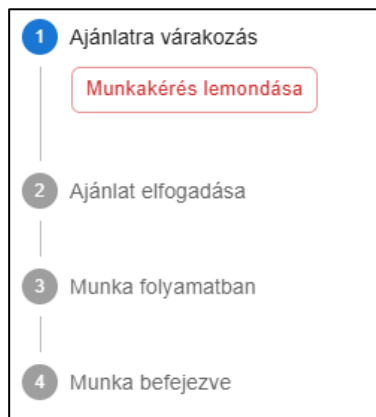
A munkák, kártyák formájában jelennek meg (7-17. sz. kép), melyeken több információ is olvasható: megnevezés, szolgáltatás, szakember/megrendelő, helyszín, leírás részlet, illetve a munka állapota (jobb felső sarok). A szakemberek esetében *Megrendelő*, az általános usereknek pedig *Szakember* attribútum van feltüntetve.

7-18. sz. kép

A kártyákra kattintással lehetőség adódik a munka minden részletének megtekintésére, illetve a munkafolyamat előre léptetésére és annak lezárására a felugró dialóguspanel segítségével (7-18. sz. kép). Ennek a fenti része, minkét felhasználói típus esetében ugyan az: itt olvasható a megbízás minden részlete (7-18. sz. kép 1). Az alatta lévő szekció (7-18. sz. kép 2) már eltér a szakember, illetve az általános felhasználók esetében. A feladata azonban egy és ugyan az: intuitívan jelezni az olvasónak, a munkafolyamaton történő áthaladás jelen állapotát, illetve az ahhoz tartozó adatokat, vagy meghatározásra váró bemeneti mezőket.

Például, új munka létrehozása után, ez lesz látható a szakember, és az általános felhasználó számára:

Általános felhasználó



7-19. sz. kép

Tisztán látható, a jelenleg aktív, illetve a következő állapotok. A jelenleg aktív nál látható a felhasználó számára felkínált egyetlen lehetőség: a „Munka lemondása”.

7-6. táblázat

Szakember felhasználó

1 Ajánlat készítése

Alapár* Ft

☐ Egységalapú árazás

Javasolt időpontok

Dátum: dd/mm/yyyy

Kezdés: --:--

Befejezés: --:--

+

Ajánlat küldése Kérés elutasítása

2 Várakozás az ajánlat elfogadására

3 Munka folyamatban

4 Munka befejezve

7-20. sz. kép

Itt több minden látható az aktív állapoton belül, ugyanis a szakembernek most kell ajánlatot adnia. Képes megadni az árazást, illetve a számára megfelelő időpontokat, majd az „Ajánlat küldése” gomb megnyomásával ezeket a megrendelővel is tudatja.

7-7. táblázat

Az ajánlat megadása után a munka átlép a következő státuszba, amit ezek a komponensek hasonlóan reflektálnak. Ez egy rövid példa volt, a teljes munkafolyamat többi

interakciójának bemutatására itt nem kerül sor. Fontos megjegyezni, hogy minden státuszban, bármely felhasználónak lehetősége van a munka lemondására.

Új megbízást létrehozni, csak az általános felhasználók tudnak, a személyes *Dashboard* felületükön, a *Munkák oldala* jobb alsó sarkában található '+' gombbal. Itt szükséges kiválasztani először egy szakembert, majd annak valamely szolgáltatását. Említésre méltó, hogy ha a felhasználó a *Kereső oldalon* vagy a szakember felhasználó *Publikus profilja* valamely alútvonalán, a „Kérvényezés” gombra kattintott, ezek a mezők alapból kitöltődnek.

8 Tesztdokumentáció

A projekt során többféle tesztelési módszert alkalmaztunk annak érdekében, hogy a rendszer megbízhatóan és hibamentesen működjön. A fejlesztési folyamatot a V-modell elvei szerint végeztük, amely biztosította, hogy minden fejlesztési lépéshez megfelelő tesztelési fázis kapcsolódjon. Ennek megfelelően nemcsak a megvalósítást, hanem már a tervezési szakaszt is folyamatosan ellenőriztük és validáltuk.

A fejlesztés során folyamatosan teszteltük az általunk írt kódot, így a hibákat már korai szakaszban sikerült kiszűrni. Az adatbázis tesztelése során többször átnéztük a struktúrát, valamint oktatói visszajelzések alapján is ellenőriztük annak helyességét.

8.1 Rendszertesztek

A frontend tesztelése főként manuálisan történt, felhasználói szemszögből, az alkalmazás funkcióinak végigkattintásával. Ennek során ellenőriztük a felhasználói felület működését és az egyes funkciók helyes viselkedését.

8.2 Egység tesztek

A backend oldalon egységteszteket készítettünk a *xUnit* keretrendszer segítségével, amelyekkel a kontrollereken belüli végpont metódusok helyes működését vizsgáltuk. Emellett integrációs tesztek is végeztünk annak ellenőrzésére, hogy az egyes modulok együttműködése megfelelő-e (pl.: *AddressService*). A backend funkciók manuális teszteléséhez a Swagger felületét használtuk, ahol az API végpontokat „kattintásos” módon próbáltuk ki.

8.2.1 AuthController

Lásd: *9.5 Egységtesztek – AuthController*

8.2.2 ExpertsController

Lásd: *9.6 Egységtesztek – ExpertsController*

8.2.3 JobsController

Lásd: *9.7 Egységtesztek – JobsController*

8.2.4 OccupationsController

Lásd: 9.8 *Egységtesztek – OccupationsController*

8.2.5 ServicesController

Lásd: 9.9 *Egységtesztek – ServicesController*

8.2.6 SettlementsController

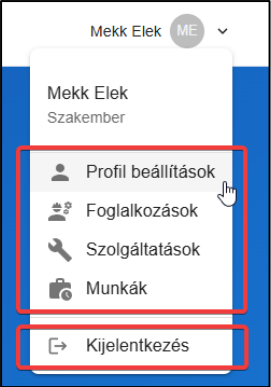
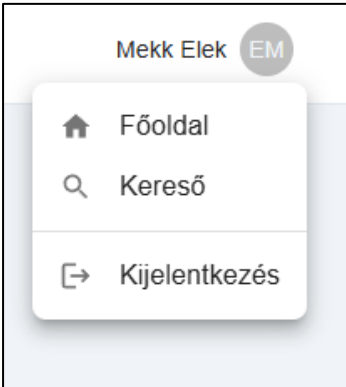
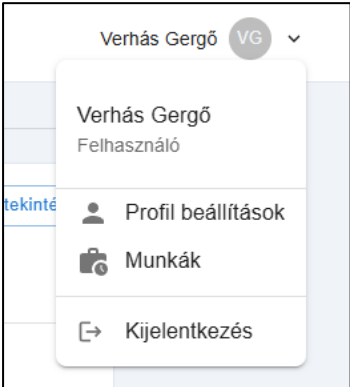
Lásd: 9.10 *Egységtesztek – SettlementsController*

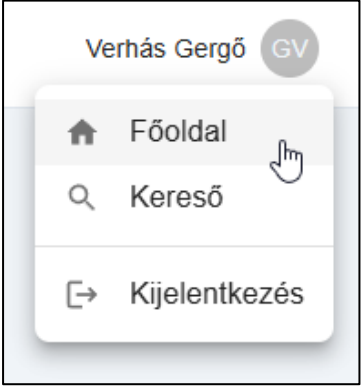
8.2.7 UsersController

Lásd: 9.11 *Egységtesztek – UsersController*

9 Mellékletek

9.1 7-5. sz. táblázat

Komponens képe	Leírás
 9-1. sz. kép	Feltétel: Bejelentkezett szakember felhasználó; kezelőfelületen (<i>dashboard/*</i>) kívül. A szakember felhasználó számára feltünteti a kezelőfelületén elérhető oldalakat. Ezekre kattintással navigálhat.
 9-2. sz. kép	Feltétel: Bejelentkezett szakember felhasználó; kezelőfelületen (<i>dashboard/*</i>) belül. A szakember felhasználó számára feltünteti a <i>Kezdő oldalra</i>, illetve a <i>Kereső oldalra</i> navigálásnak lehetőségét.
 9-3. sz. kép	Feltétel: Bejelentkezett általános felhasználó; kezelőfelületen (<i>dashboard/*</i>) kívül. Az általános felhasználó számára feltünteti a kezelőfelületén elérhető oldalakat. Ezekre kattintással navigálhat.

 9-4. sz. kép	Feltétel: Bejelentkezett általános felhasználó; kezelőfelületen (<i>dashboard/*</i>) belül. A szakember felhasználó számára feltűnteti a <i>Kezdő oldalra</i>, illetve a <i>Kereső oldalra</i> navigálásnak lehetőségét.
---	---

9-1. táblázat

9.2 Er-diagram

9.3 Fő dump települései

Irányítószám	Település	Vármegye
8127	Aba	Fejér
5241	Abádszalók	Jász-Nagykun-Szolnok
7678	Abaliget	Baranya
3261	Abasár	Heves
3882	Abaújalpár	Borsod-Abaúj-Zemplén
3882	Abaújkér	Borsod-Abaúj-Zemplén
3815	Abaújlak	Borsod-Abaúj-Zemplén
3881	Abaújszántó	Borsod-Abaúj-Zemplén
9700	Szombathely	Vas
9600	Sárvár	Vas
9798	Ják	Vas
8019	Székesfehérvár	Fejér
4042	Debrecen	Hajdú-Bihar
8308	Zalahaláp	Veszprém
8300	Tapolca	Veszprém

9-2. táblázat

9.4 Felhasználók listája

Minden előre feltöltött profil jelszava: „asdf1234A!”

Név	Email	Típus
Verhás Gergő	verhas.gergo@hbsz.edu.hu	Általános felhasználó
Varga Dávid	varga.david@hbsz.edu.hu	Általános felhasználó
Császár Bálint	csaszar.balint@hbsz.edu.hu	Általános felhasználó
Teszt Elek	teszt.elek@gmail.com	Általános felhasználó
Horváth Péter	horvath.peter@gmail.com	Általános felhasználó
Molnár Andrea	molnar.andrea@gmail.com	Általános felhasználó
Mekk Elek	mekk.elek@ezermester.hu	Szakember
Csavaros Csaba	csavaros.csaba@gmail.com	Szakember
Kalapács Karcsi	kalapacs.karcsi@ezermester.hu	Szakember
Flexes Feri	flexes.feri@gmail.com	Szakember
Csőtörő Csöpi	csotoro.csopi@vizman.hu	Szakember
Szigszalagos Sanyi	szigszalagos.sanyi@gmail.com	Szakember

9-3. táblázat

9.5 Egységtesztek – AuthController

Teszt neve	Vizsgálat	Elvárt eredmény
Login_Success_ReturnsOk	Helyes bejelentkezési adatok	200 OK válasz, a rendszer hitelesíti a felhasználót.
Login_UserNotFound_ReturnsNotFound	Nem létező felhasználó	404 NotFound válasz, a rendszer nem találja a megadott felhasználót
Login_WrongPassword_ReturnsUnauthorized	Hibás jelszó	401 Unauthorized válasz, a jelszó ellenőrzés sikertelen
Register_DuplicateEmail_ReturnsBadRequest	Már létező email	400 BadRequest válasz, a rendszer elutasítja a regisztrációt, mert az email már használatban van
Register_Success_ReturnsOk	Érvényes regisztráció	200 OK válasz, a felhasználó létrejön az adatbázisban és automatikusan hozzárendelődik a "Generic" szerepkörhöz

Register_WeakPassword_Returns BadRequest	Gyenge jelszó	400 BadRequest válasz, a rendszer elutasítja a nem megfelelő erősségű jelszót
---	---------------	---

9.6 Egységtesztek – ExpertsController

Teszt neve	Vizsgálat	Elvárt eredmény
CreateExpertSpecialty_NewOccupation_ReturnsCreated	Új szakma létrehozása	201 Created válasz, az új szakma elmentésre kerül az adatbázisban
CreateService_ValidData_ReturnsCreated	Szolgáltatás létrehozása	201 Created válasz, a szolgáltatás létrejön és a szakemberhez kerül hozzárendelésre
DeleteService_NotOwned_ReturnsNotFound	Nem saját szolgáltatás törlése	404 NotFound válasz, a rendszer nem találja a felhasználóhoz tartozó szolgáltatást
GetExperts_FilterByCity_ReturnsCorrectExpert	Szűrés város szerint	200 OK válasz, a rendszer csak a megadott városban dolgozó szakembereket adja vissza
GetExperts_FilterByOccupation_ReturnsOnlyRelevant	Szűrés szakma szerint	200 OK válasz, csak a megadott szakmához tartozó szakemberek kerülnek visszaadásra
GetMe_ExpertExists_ReturnsOk	Saját szakember profil lekérése	200 OK válasz, a rendszer visszaadja a felhasználóhoz tartozó szakember adatokat
GetMe_NotExpert_ReturnsNotFound	Nem szakember felhasználó	404 NotFound válasz, nincs szakember profil

9.7 Egységtesztek – JobsController

Teszt neve	Vizsgálat	Elvárt eredmény
------------	-----------	-----------------

AcceptOffer_CallerNotInitiator_ReturnsForbid	Nem kezdeményező fogad el ajánlatot	403 Forbidden válasz, a művelet nem engedélyezett
AcceptOffer_JobNotOffered_ReturnsConflict	Nem ajánlott job elfogadása	409 Conflict válasz, a job nincs ajánlott állapotban
AcceptOffer_ValidAcceptance_ReturnsAccepted	Ajánlat elfogadása	200 OK válasz, a job állapota elfogadottra változik
CancelJob_CallerNotParty_ReturnsForbid	Nem érintett fél töröl	403 Forbidden válasz, nincs jogosultság
CancelJob_ExpertCancelsOffered_ReturnsCancelled	Szakember törli	200 OK válasz, a job törölt állapotba kerül
CancelJob_InitiatorCancelsPending_ReturnsCancelled	Kezdeményező törli	200 OK válasz, a job törölt állapotba kerül
CancelJob_JobAlreadyCompleted_ReturnsConflict	Már befejezett job törlése	409 Conflict válasz, a művelet nem hajtható végre
CompleteJob_AfterOfferedTime_ReturnsCompleted	Idő után befejezés	200 OK válasz, a job completed állapotba kerül
CompleteJob_CallerNotExpert_ReturnsForbid	Nem szakember fejez be	403 Forbidden válasz
CompleteJob_JobNotAccepted_ReturnsConflict	Nem elfogadott job	409 Conflict válasz
CompleteJob_TimeNotPassed_ReturnsBadRequest	Idő még nem telt le	400 BadRequest válasz
CreateJob_DuplicateAddress_ReturnsExistingRecord	Azonos cím	201 Created válasz, a rendszer meglévő címet használ
CreateJob_EmptyTitle_ReturnsBadRequest	Üres cím	400 BadRequest válasz
CreateJob_EndBeforeStart_ReturnsBadRequest	Hibás idő	400 BadRequest válasz
CreateJob_NoTimeIntervals_ReturnsBadRequest	Nincs időintervallum	400 BadRequest válasz
CreateJob_OverlappingIntervals_ReturnsBadRequest	Átfedés	400 BadRequest válasz

CreateJob_ServiceNotFound_ReturnsNotFound	Szolgáltatás nem létezik	404 NotFound válasz
CreateJob_SettlementNotFound_ReturnsNotFound	Település nem létezik	400 BadRequest válasz
CreateJob_ValidDto_ReturnsCreated	Érvényes job	201 Created válasz, a job elmentésre kerül
GetJob_CallerIsExpert_ReturnsOk	Szakember lekéri	200 OK válasz
GetJob_CallerIsInitiator_ReturnsOk	Kezdeményező lekéri	200 OK válasz
GetJob_CallerNotParty_ReturnsForbidden	Nem érintett lekéri	403 Forbidden válasz
GetJob_JobNotFound_ReturnsNotFound	Nem létező job	404 NotFound válasz
MakeOffer_CallerIsInitiator_ReturnsConflict	Kezdeményező ajánlatot tesz	409 Conflict válasz
MakeOffer_CallerNotExpert_ReturnsForbidden	Nem szakember tesz ajánlatot	403 Forbidden válasz
MakeOffer_EndBeforeStart_ReturnsBadRequest	Hibás idő	400 BadRequest válasz
MakeOffer_JobNotPending_ReturnsConflict	Nem megfelelő állapot	409 Conflict válasz
MakeOffer_NoTimeIntervals_ReturnsBadRequest	Nincs idő	400 BadRequest válasz
MakeOffer_OverlappingIntervals_ReturnsBadRequest	Átfedés a megadott időintervallumok között	400 BadRequest válasz
MakeOffer_ValidOffer_ReturnsOk	Érvényes ajánlat	200 OK válasz, a job offered állapotba kerül
RateJob_AlreadyRated_ReturnsConflict	Már értékelt	409 Conflict válasz

RateJob_CallerNotInitiator_ReturnsForbidden	Nem kezdeményező értékel	403 Forbidden válasz
RateJob_JobNotCompleted_ReturnsConflict	Nem completed job	409 Conflict válasz
RateJob_RatingTooHigh_ReturnsBadRequest	Túl magas érték	400 BadRequest válasz
RateJob_RatingTooLow_ReturnsBadRequest	Túl alacsony érték	400 BadRequest válasz
RateJob_ValidRating_SetsRating	Érvényes értékelés	200 OK válasz, az értékelés mentésre kerül az adatbázisban

9.8 Egységtesztek – OccupationsController

Teszt neve	Vizsgálat	Elvárt eredmény
GetOccupations_ReturnsAllOccupations	Összes szakma lekérése	200 OK válasz, az összes szakma listázásra kerül

9.9 Egységtesztek – ServicesController

Teszt neve	Vizsgálat	Elvárt eredmény
GetPricingOptions_NoServices_ReturnsDefaults	Nincs adat	200 OK válasz, alapértelmezett értékek kerülnek visszaadásra
GetPricingOptions_ReturnsCorrectAggregates	Aggregáció	200 OK válasz, helyes statisztikai adatok kerülnek visszaadásra
GetTruncated_ExpertWithNoServices_ReturnsEmptyList	Nincs szolgáltatás	200 OK válasz, a rendszer üres listát ad vissza
GetTruncated_SpecificExpert_ReturnsOnlyTheirServices	Szűrés szakember szerint	200 OK válasz, csak a megadott szakemberhez tartozó szolgáltatások kerülnek visszaadásra

9.10 Egységtesztek – SettlementsController

Teszt neve	Vizsgálat	Elvárt eredmény
GetByCounty_DistinctNamesOnly	Duplikációk szűrése	200 OK válasz, csak egyedi nevek

GetByCounty_InvalidCounty_ReturnsEmpty	Hibás vármegye	200 OK válasz, üres lista
GetByCounty_ReturnsCorrectSettlements	Lekérés	200 OK válasz, helyes adatok
GetCounties_NoExperts_ReturnsEmptyList	Nincs adat	200 OK válasz, üres lista
GetCounties_ReturnsDistinctOrderedCounties	Rendezés	200 OK válasz, rendezett lista
GetCounties_SkipsNullLocations	Null kezelés	200 OK válasz, null értékek kihagyva

9.11 Egységtesztek – UsersController

Teszt neve	Vizsgálat	Elvárt eredmény
GetMe_NoAuth_ReturnsUnauthorized	Nincs bejelentkezés	401 Unauthorized válasz
GetMe_ReturnsOwnData_IfAuthorized	Saját adatok	200 OK válasz, a felhasználó adatai visszaadásra kerülnek
UpdateMe_IdentityError_ReturnsBadRequest	Identity hiba	400 BadRequest válasz
UpdateMe_UserNotFound_ReturnsNotFound	Nem létező user	404 NotFound válasz
UpdateMe_ValidData_ReturnsOkAndUpdatesUser	Módosítás	200 OK válasz, a felhasználó adatai frissítésre kerülnek az adatbázisban